Deep Learning Framework for the Multi-Disease Diagnosis of Heart Disease, Pneumonia, and Diabetic Retinopathy Using ResNet, MobileNet and DenseNet

TABLE OF CONTENTS

CH No	Title	Page No
	Table of Contents	i
	List of Figures	iv
	List of Tables	vi
	Nomenclature	vii
	Abstract	viii
1	**INTRODUCTION**	**1**
	1.1 Background of the Project	1
	1.2 Problem Statement	2
	1.3 Objective of the Project	2
	1.4 Scope of the Project	2
	1.5 Methodology Overview	3
	1.6 Organization of the Report	4
2	**LITERATURE REVIEW**	**6**
	2.1 Previous Research and Related Work	6
	2.2 Existing Solutions and Their Limitations	10
	2.3 Gap Analysis	11
	2.4 Relevance of the Project	12
3	**SYSTEM ANALYSIS**	**13**
	3.1 Requirement Analysis	13
	3.1.1 Functional Requirements	13
	3.1.2 Non-functional Requirements	14
	3.2 Feasibility Study	14

3.2.1 Technical Feasibility 15

3.2.2 Economic Feasibility 15

3.2.3 Social Feasibility 15

3.3 Proposed System Overview 16

4 SYSTEM DESIGN 17

4.1 System Architecture 17

4.2 Block Diagram 17

4.3 Data Flow Diagram 18

4.4 UML Diagram 19

4.4.1 Use Case Diagram 19

4.4.2 Class Diagram 20

4.4.3 Sequence Diagram 21

4.4.4 Activity Diagram 22

4.5 Database Design 23

5 IMPLEMENTATION 26

5.1 Programming Languages and Technologies Used 26

5.2 Development Tools and Environments 34

5.3 Module-wise Implementation Details 36

5.3.1 Data Acquisition and Preprocessing Module 36

5.3.2 Model Development Module 37

5.3.3 Training and Evaluation Module 37

5.3.4 User Interface Module 37

5.3.5 Deployment and Maintenance Module 38

5.4 Algorithms and Logic Used 38

5.4.1 MobileNet 38

5.4.2 DenseNet 39

5.4.3 ResNet 40

5.4.4 CNN 40

6 TESTING AND RESULTS 43

6.1 Testing Methodologies 43

6.1.1 Unit Testing 43

6.1.2 Integration Testing 43

6.1.3 System Testing 44

6.2 Test Cases and Reports 45

6.3 Performance Evaluation 46

6.4 Screenshots of Application Output 50

7 CONCLUSION AND FUTURE WORK 57

7.1 Summary of Findings 57

7.2 Key Achievements and Contributions 57

7.3 Challenges Faced 58

7.4 Future Scope and Improvements 58

8 REFERENCES 60

LIST OF FIGURES

Figure No.	Figure Name	Page No.
1.5.1	Waterfall Model	3
4.1.1	System Architecture Diagram	17
4.2.1	Block Diagram	18
4.3.1	DFD Diagram	19
4.4.1.1	Use Case Diagram	20
4.4.2.1	Class Diagram	21
4.4.3.1	Sequence Diagram	22
4.4.4.1	Activity Diagram	22
4.5.1	Tables created for this Project	23
4.5.2	Structure of Admin Table	23
4.5.3	Structure of Audit_log table	24
4.5.4	Structure of OTP Table	24
4.5.5	Structure of Prediction Table	24
4.5.6	Structure of User Table	25
5.4.1.1	MobileNet Architecture	39
5.4.2.1	DenseNet Block	39
5.4.3.1	ResNet Architecture	40
5.4.4.1	CNN Architecture	41
6.3.1	Classification report for Pneumonia Prediction	46
6.3.2	Confusion Matrix for Pneumonia Prediction	46
6.3.3	Classification Report for Heart Disease Prediction	47
6.3.4	Confusion Matric for Heart Disease Prediction	47

6.3.5	Classification Report for DR Prediction	48
6.3.6	Classification Report for Diabetes Prediction	49
6.3.7	Confusion Matrix for Diabetes Prediction	49
6.4.1	Homepage	50
6.4.2	Services of the Project	50
6.4.3	Services of the Project	51
6.4.4	User guide to go through the services	51
6.4.5	Heart Disease Prediction	52
6.4.6	Diabetes Prediction	53
6.4.7	Pneumonia Prediction	54
6.4.8	Diabetic Retinopathy Prediction	55
6.4.9	User Dashboard	56
6.5.10	Admin Dashboard	56

LIST OF TABLES

Table No.	Table Name	Page No.
6.2.1	Testing Report	45

NOMENCLATURE

Acronym	Full Form	Description
AI	Artificial Intelligence	Simulation of human intelligence in machines to perform tasks such as classification and recognition.
ML	Machine Learning	A subset of AI that enables systems to learn from data and improve classification accuracy over time.
DL	Deep Learning	A branch of ML that uses artificial neural networks to process complex patterns, such as image classification.
CNN	Convolutional Neural Network	A type of deep learning model specialized in analysing visual data for classification and recognition.
KNN	K-nearest neighbor	One of the simplest forms of supervised ML algorithm that is used for both classification and regression problems.
DR	Diabetic Retinopathy	a serious eye condition caused by damage to the blood vessels in the retina (the light-sensitive tissue at the back of the eye) due to diabetes, potentially leading to vision loss and blindness if left untreated.
UI	User Interface	The graphical interface through which users interact with the plant identification system.
F1 Score	F1 Score	A metric that combines precision and recall to provide a balanced measure of model performance.
DenseNet	Densely Connected Convolutional Networks	A deep learning architecture that addresses the vanishing gradient problem in CNNs by densely connecting each layer to every other layer, enabling feature reuse and improved gradient flow.
ResNet	Residual Neural Network	A deep learning architecture designed to improve feature extraction and classification accuracy.
MobileNet	MobileNet	Is a family of CNN architectures designed for image classification, object detection, and other computer vision tasks.

ABSTRACT

The application of deep learning in medical diagnosis has shown remarkable advancements in improving healthcare solutions. This study focuses on the detection and classification of three critical diseases: heart disease, pneumonia, and diabetic retinopathy (DR). Each disease presents unique diagnostic challenges, including variability in data representation and the need for high precision. The proposed framework incorporates CNN, Resnet, MobileNet and DenseNet architectures to build a robust system capable of addressing these challenges. Using the UCI Heart Disease dataset, chest X-ray images for pneumonia detection, and color fundus images for DR classification, the model achieves significant accuracy improvements over traditional methods. For pneumonia, the VGG16-based neural network demonstrates strong baselines, achieving an accuracy of 97% on a diverse dataset. Similarly, the DR detection system integrates convolutional neural networks to identify lesions effectively. By implementing CNN, Resnet, MobileNet and DenseNet, this research enhances computational efficiency and diagnostic accuracy. Comparative analysis with existing systems, including KNN, SVM, Random Forest, and Naïve Bayes, highlights the superiority of the proposed approach. The study demonstrates that the hybrid architecture achieves higher accuracy, reduced computational cost, and faster processing time, making it suitable for real-time clinical applications.

Keywords: Deep Learning, MobileNet, DenseNet, ResNet, CNN, Heart Disease Detection, Pneumonia Diagnosis.

Chapter 1: Introduction

1.1 Background of the Project

The rapid advancement of deep learning has revolutionized the field of medical diagnostics, offering highly accurate and efficient solutions for disease detection and classification. Early diagnosis of critical conditions such as heart disease, pneumonia, and diabetic retinopathy (DR) is essential for timely intervention and improved patient outcomes. However, traditional diagnostic methods rely heavily on manual interpretation, which can be time-consuming and prone to errors. Deep learning-based medical image analysis provides a transformative approach by automating disease detection, reducing diagnostic errors, and enabling real-time clinical applications.

This study focuses on developing a robust deep learning framework that integrates ResNet, MobileNet, and DenseNet architectures to enhance the accuracy and efficiency of disease classification. The proposed system utilizes multiple medical datasets, including the UCI Heart Disease dataset for cardiac conditions, chest X-ray images for pneumonia detection, and color fundus images for DR classification. By leveraging the strengths of deep convolutional neural networks (CNNs), the system effectively extracts and learns essential features from medical images, ensuring a high level of precision in disease detection.

The study also explores comparative analysis with traditional machine learning techniques, including K-Nearest Neighbors (KNN), Support Vector Machines (SVM), Random Forest, and Naïve Bayes. These conventional models, while useful, often struggle with complex medical image patterns and require extensive feature engineering. In contrast, the hybrid deep learning approach presented in this study achieves superior accuracy, reduced computational cost, and faster processing times. Notably, the VGG16-based neural network achieves a 97% accuracy rate in pneumonia classification, demonstrating the effectiveness of deep learning in medical applications.

By integrating hyperparameter optimization and advanced CNN architectures, the proposed system enhances real-time diagnostic capabilities, making it well-suited for clinical environments. The research underscores the potential of deep learning to revolutionize medical diagnostics by providing scalable, efficient, and highly accurate disease detection models. As healthcare continues to evolve with AI-driven solutions,

this study contributes to bridging the gap between technology and medicine, paving the way for more automated, accessible, and reliable diagnostic tools.

1.2 Problem Statement

Heart disease, pneumonia, and diabetic retinopathy pose significant global health burdens, requiring timely and accurate diagnosis for effective treatment. However, conventional diagnostic approaches struggle with variability in data representation, complex feature extraction, and limited generalization across diverse datasets. Existing machine learning techniques such as KNN, SVM, and Random Forest lack the deep feature extraction capabilities required for high-precision medical image classification. To address these limitations, this study proposes a deep learning framework integrating CNN architectures—ResNet, MobileNet, and DenseNet—to enhance disease classification accuracy while reducing computational complexity. By utilizing medical datasets, including the UCI Heart Disease dataset, chest X-ray images, and color fundus images, this research aims to develop a robust, automated system for real-time clinical applications.

1.3 Objective of the Project

To develop an efficient and accurate deep learning-based system for diagnosing heart disease, pneumonia, and diabetic retinopathy. The proposed system leverages ResNet, CNN, MobileNet and DenseNet architectures to classify and detect these diseases using heterogeneous datasets, thereby improving diagnostic performance and reducing the limitations of existing methods.

1.4 Scope of the Project

The rapid advancements in deep learning have significantly transformed medical diagnostics, offering enhanced accuracy and efficiency in disease detection. Traditional diagnostic methods often rely on manual interpretation, which can be time-consuming, prone to human error, and highly dependent on expertise availability. With increasing cases of heart disease, pneumonia, and diabetic retinopathy (DR), there is a pressing need for automated, reliable, and scalable diagnostic tools. Convolutional Neural Networks (CNNs) have demonstrated remarkable success in medical image analysis, making them an ideal solution for this challenge. By leveraging the capabilities of ResNet, MobileNet, and DenseNet, this study aims to enhance computational

efficiency, improve classification accuracy, and provide real-time diagnostic support for critical diseases.

1.5 Methodology Overview

Software Development Life Cycle

In our project we use waterfall model as our software development cycle because of its step-by-step procedure while implementing.

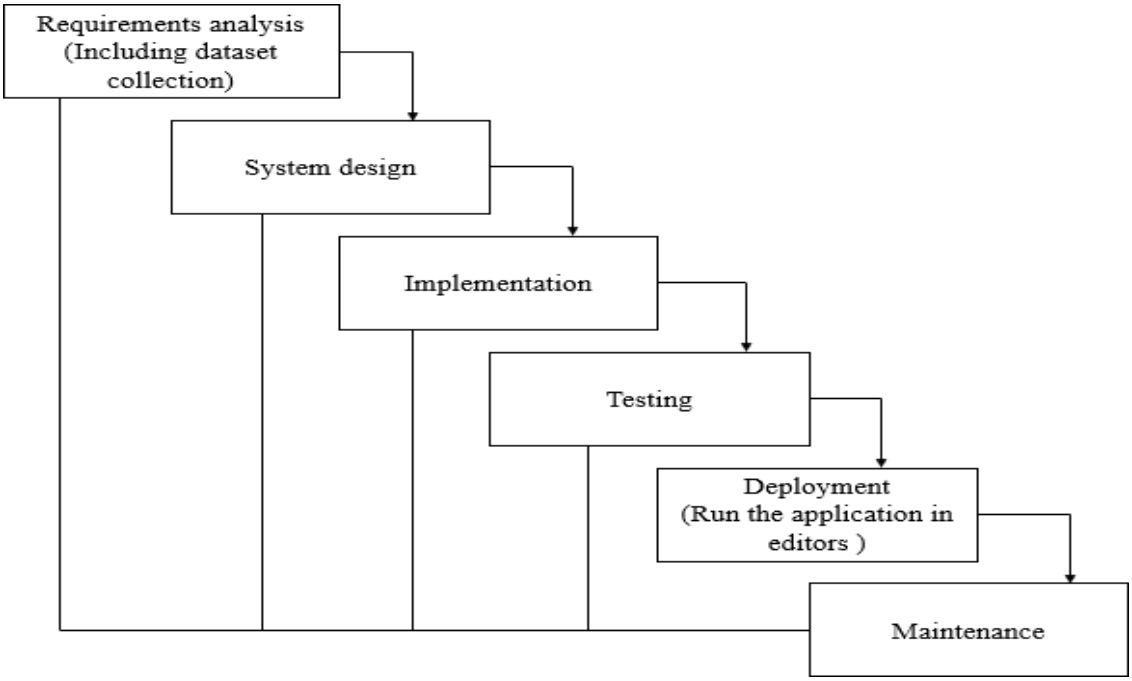

Fig 1.5.1: Waterfall Model

- **Requirement Gathering and analysis** − All possible requirements of the system to be developed are captured in this phase and documented in a requirement specification document.
- **System Design** − the requirement specifications from first phase are studied in this phase and the system design is prepared. This system design helps in specifying hardware and system requirements and helps in defining the overall system architecture.
- **Implementation** − with inputs from the system design, the system is first developed in small programs called units, which are integrated in the next phase. Each unit is developed and tested for its functionality, which is referred to as Unit Testing.

- **Integration and Testing** − All the units developed in the implementation phase are integrated into a system after testing of each unit. Post integration the entire system is tested for any faults and failures.
- **Deployment of system** − Once the functional and non-functional testing is done; the product is deployed in the customer environment or released into the market.
- **Maintenance** − There are some issues which come up in the client environment. To fix those issues, patches are released. Also, to enhance the product some better versions are released. Maintenance is done to deliver these changes in the customer environment.

1.6 Organization of the Report

The report for the "Deep Learning Framework for the Multi-Disease Diagnosis of Heart Disease, Pneumonia, and Diabetic Retinopathy Using ResNet, MobileNet and DenseNet" project is organized as follows:

1. **Chapter 1: Introduction**

Introduces the project, outlining the problem, objectives, background, and scope, with a focus on its relevance for tech workers in high-stress environments.

2. **Chapter 2: Literature Review**

Reviews existing mental health diagnosis systems, identifies limitations, and highlights the contribution of this project.

3. **Chapter 3: System Analysis**

Discusses functional and non-functional requirements, feasibility, and provides an overview of the proposed system focusing on scalability and performance.

4. **Chapter 4: System Design**

Describes the technical design, including system architecture, data flow, and database design with visual diagrams.

5. **Chapter 5: Implementation**

Details the technologies and tools used, along with the process of model training, feature extraction, and system integration.

6. **Chapter 6: Testing and Results**

Explains the testing methodologies and presents key performance metrics like accuracy, precision, recall, and F1-score.

7. **Chapter 7: Conclusion and Future Work**

Summarizes key findings, discusses challenges, and suggests improvements and potential future research.

8. **Chapter 8: References**

Lists all cited sources in IEEE or APA style.

Chapter 2: Literature Review

2.1 Previous Research and Related Work

1. Systematic Review on Deep Learning-based Heart Disease Diagnosis

Authors: V. C A and V. Baby Shalini

Published on: 16 August 2023

DOI: 10.1109/ICECAA58104.2023.10212392

This review study systematically examines research papers published between 2017 and 2022 that focus on heart disease detection using deep learning. The methodology involves an extensive literature survey of various deep learning techniques applied to cardiovascular disease prediction, including convolutional neural networks (CNNs), recurrent neural networks (RNNs), and hybrid models integrating data mining approaches. The collected studies are analyzed based on key performance metrics such as accuracy, sensitivity, specificity, and computational efficiency. Additionally, comparisons are made among different models to assess their effectiveness in identifying hypertensive and vascular heart diseases. The review also explores how data preprocessing, feature extraction, and hyperparameter optimization contribute to improving the diagnostic performance of deep learning models.

Despite the advancements in deep learning for heart disease detection, several limitations persist. One major challenge is the variability in datasets, as models trained on specific populations may not generalize well to diverse demographics.

Additionally, many studies rely on relatively small datasets, which can lead to overfitting and reduced model reliability. The interpretability of deep learning models remains another concern, as their "black-box" nature makes it difficult for medical professionals to trust and validate predictions. Furthermore, computational complexity and resource-intensive training processes can hinder real-time deployment in clinical settings. Lastly, the integration of these models into healthcare systems requires regulatory approvals and validation through extensive clinical trials, which can slow down adoption.

2. Deep Learning Techniques for Diabetic Retinopathy Classification

Authors: M. Z. Atwany, A. H. Sahyoun and M. Yaqub

Published on: 08 March 2022

DOI: 10.1109/ACCESS.2022.3157632

This study systematically reviews and analyzes deep learning-based approaches for the detection and classification of Diabetic Retinopathy (DR) using retinal fundus images. The methodology involves evaluating state-of-the-art supervised, self-supervised, and Vision Transformer models for DR classification, including referable, non-referable, and proliferative DR stages. The paper explores existing datasets such as EyePACS, Messidor, and APTOS, assessing their suitability for training and validating deep learning models. Various image preprocessing techniques, such as contrast enhancement, denoising, and segmentation, are examined to improve feature extraction. The study also investigates different convolutional neural network (CNN) architectures, including ResNet, DenseNet, and MobileNet, along with attention-based mechanisms in Vision Transformers. Performance metrics such as accuracy, sensitivity, specificity, and AUC-ROC scores are used to compare the effectiveness of different models in detecting early-stage DR.

Despite advancements in deep learning for DR detection, several challenges persist. One of the primary limitations is the dependency on high-quality labeled datasets, which require expert ophthalmologists for annotation, making data collection time-consuming and expensive. Additionally, variations in image quality, lighting conditions, and retinal abnormalities can impact model performance and lead to misclassification. Another significant challenge is the interpretability of deep learning models, as their decision-making process is often considered a "black box," limiting trust and clinical adoption. Furthermore, the generalizability of these models across diverse populations remains an issue, as models trained on specific datasets may not perform well on images from different ethnicities or geographic regions. Lastly, real-world deployment of DR detection systems requires regulatory approvals, seamless integration with clinical workflows, and validation through extensive clinical trials, which can slow down large-scale implementation.

3. Heart Disease Diagnosis using Deep Learning

Authors: K. Mehta and K. Subramanian,

Published on: 24 August 2022

DOI: 10.1109/INDISCON54605.2022.9862847

The proposed study focuses on automating the calculation of Left Ventricle (LV) Ejection Fraction from cardiac MRI scans using image processing and deep learning techniques. The process begins with preprocessing DICOM (Digital Imaging and Communications in Medicine) images by converting them into a more accessible PNG format. Image enhancement techniques, such as noise reduction and contrast adjustment, are applied to improve segmentation accuracy. Segmentation of the LV is then performed using deep learning-based models like U-Net or traditional contour-based segmentation techniques. After extracting the LV region, the end-diastolic volume (EDV) and end-systolic volume (ESV) are computed. The ejection fraction (EF) is then calculated using the formula:

EF=((EDV−ESV)/EDV)×100EF=\frac{EDV-ESV}{EDV}\times 100EF=EDVEDV−ESV×100

This approach minimizes manual effort, providing cardiologists with a fast and accurate tool for cardiac function assessment. Despite its advantages, the automated EF calculation system faces several challenges. First, MRI scans exhibit significant variability in quality, resolution, and contrast, which may affect segmentation accuracy. Ensuring the model generalizes well across different patient datasets and MRI machines remains a challenge. Additionally, deep learning-based segmentation models require a large volume of labeled training data, which can be time-consuming to obtain. Another limitation is the potential misidentification of the LV boundary due to overlapping structures or artifacts in the images, leading to minor errors in EF computation. Finally, real-world clinical deployment requires extensive validation through multi-center trials, regulatory approvals, and integration into hospital information systems, which can delay widespread adoption.

4. Simultaneous Diagnosis of Severity and Features of Diabetic Retinopathy in Fundus Photography Using Deep Learning

Authors: J. Wang, Y. Bai and B. Xia

Published on: 28 July 2020

DOI: 10.1109/JBHI.2020.3012547

The study presents a hierarchical multi-task deep learning framework for diagnosing diabetic retinopathy (DR) and detecting DR-related features from fundus images. The proposed approach integrates a hierarchical structure that incorporates the causal relationships between DR-related features (such as microaneurysms, hemorrhages, and exudates) and DR severity levels. The framework consists of a convolutional neural network (CNN) backbone, which extracts feature representations from retinal fundus images, followed by a multi-task learning model that simultaneously predicts both DR severity and specific DR-related features. The hierarchical structure ensures that feature detection aids severity classification, enhancing interpretability. The performance of the model is evaluated using quadratic weighted Cohen's kappa coefficient, receiver operating characteristic (ROC) analysis, and precision-recall analysis. Additionally, a grader study compares the model's performance with general ophthalmologists at varying experience levels.

Despite its advantages, the proposed approach faces several challenges. First, the reliance on high-quality annotated datasets for both DR severity levels and DR-related features makes model training time-consuming and resource-intensive. Additionally, while the hierarchical structure improves interpretability, the introduction of multiple prediction tasks increases computational complexity, potentially requiring significant processing power and storage. Another limitation is the generalizability of the model across different populations and imaging conditions, as variations in fundus image quality and patient demographics could affect performance. Finally, clinical implementation requires rigorous validation through large-scale multi-center studies, regulatory approvals, and integration with existing ophthalmology diagnostic workflows, which may delay its adoption in real-world clinical settings.

5. A Deep Learning Ensemble Approach for Diabetic Retinopathy Detection

Authors: S. Qummar et al.

Published on: 15 October 2019

DOI: 10.1109/ACCESS.2019.2947484

This study proposes an ensemble deep learning approach for the automatic detection and classification of Diabetic Retinopathy (DR) stages from retinal fundus

images. The research utilizes a publicly available Kaggle dataset of retina images to train and evaluate five deep convolutional neural network (CNN) models: ResNet50, InceptionV3, Xception, DenseNet121, and DenseNet169. The ensemble model aggregates the predictions from these architectures, leveraging their unique feature extraction capabilities to improve classification accuracy. Each CNN model processes the images independently, extracting complex features, and the final classification is performed by combining their outputs using an ensemble strategy such as majority voting or weighted averaging. The proposed method aims to enhance the detection of early-stage DR, addressing the limitations of previous approaches with low sensitivity to mild and moderate DR cases. The experimental evaluation includes performance metrics such as accuracy, precision, recall, F1-score, and area under the receiver operating characteristic (ROC) curve to assess the effectiveness of the ensemble model.

Despite its improved classification performance, the proposed ensemble deep learning approach has several limitations. First, training multiple deep CNN models significantly increases computational requirements, making real-time implementation on standard clinical hardware challenging. Additionally, deep learning models are highly dependent on large, well-annotated datasets, and any inconsistencies or biases in the Kaggle dataset could impact the generalizability of the model to real-world clinical settings. Another limitation is the black-box nature of deep learning models, which can make it difficult for clinicians to interpret the decisions made by the ensemble model, potentially limiting its adoption in medical practice. Furthermore, the variability in fundus image quality, illumination conditions, and patient demographics may affect model performance, necessitating additional preprocessing steps or domain adaptation techniques to improve robustness. Finally, rigorous clinical validation and integration with existing ophthalmology diagnostic systems would be required before deployment in real-world healthcare environments.

2.2 Existing Solutions and Their Limitations

1.Heart Disease Diagnosis: Traditional machine learning algorithms like KNN, SVM, Naïve Bayes, and Random Forest are employed for heart disease prediction using the UCI dataset.

2.Pneumonia Detection: VGG16 combined with Neural Networks is used to classify pneumonia on chest X-ray images.

3.Diabetic Retinopathy Detection: Convolutional neural networks (CNNs) process fundus retina images for lesion identification and classification.

Limitations of Existing Systems:

- Limited efficiency due to manual hyperparameter tuning and complex architectures.
- High computational costs and extended training times in traditional CNNs like VGG16.
- Lower accuracy and recall metrics when applied to diverse datasets.
- Inability to handle real-time diagnosis effectively due to resource-intensive models.
- Susceptibility to overfitting in small datasets.

2.3 Gap Analysis

The current medical diagnosis systems lack necessary elements so researchers are developing deep learning-based technology as a solution. Manual interpretation of traditional diagnostic methods requires extensive periods of time and leads to errors and depends entirely on available experts. The inconsistent methods used for data display introduce additional complexities to accuracy and consistency measurements. KNN and SVM along with Random Forest experience difficulties in deep feature extraction leading to challenging requirements for extensive feature engineering and often produce inaccurate results. The current deep learning algorithms encounter various operational problems because they possess intricate system structures together with expensive computation requirements and little fit between small datasets as well as slow response times in medical diagnosis. The proposed system tackles these problems through the integration of advanced deep learning architectural systems to improve accuracy levels. The system simplifies computations while keeping diagnosis reliability intact. Precision and efficiency balance in the system comes from the combination of MobileNet with ResNet and DenseNet architectures. The developed system helps achieve faster real-time medical diagnostics. The system targets a set of scalable diagnostic tools with automated processes and universal accessibility for diagnosing diseases including heart disease and pneumonia and diabetic retinopathy.

2.4 Relevance of the Project

The project extends the work of prior deep learning medical diagnosis studies to resolve drawbacks in classic approaches and current machine learning practices. Recent medical research about heart disease along with diabetic retinopathy and additional applications is integrated to boost accuracy along with efficiency levels. The project foundation draws from thorough research evaluation which shows its value for ongoing progress. The project implements three proven medical datasets derived from UCI Heart Disease information and Chest X-ray and Color fundus pictures to build trustworthy evaluation frameworks. The system uses contemporary deep learning architectural elements that integrate CNNs with ResNet and MobileNet and DenseNet along with VGG16. The models deliver advanced diagnostic capabilities while extracting features with high reliability. The project achieves better efficiency and disease classification precision through the incorporation of state-of-the-art approaches. The system applies to academic research as well as practical healthcare needs. This system functions to eliminate current diagnostic limitations while enhancing speed of real-time evaluation through its approach. The research develops advanced methods that enable automatic medical diagnostics to scale while ensuring accessibility for all systems.

Chapter 3: System Analysis

3.1 Requirement Analysis

3.1.1 Functional Requirements

- **Disease Classification System**

  The system must classify heart disease, pneumonia, and diabetic retinopathy (DR) using deep learning models. It must distinguish between different severity levels of DR using color fundus images.

- **Deep Learning Model Integration**

  The system should integrate CNN, ResNet, MobileNet, and DenseNet architectures for disease detection and classification. It should utilize VGG16 for pneumonia detection, achieving high accuracy in chest X-ray image analysis.

- **Dataset Utilization**

  The system must use the UCI Heart Disease dataset for heart disease classification. It should process chest X-ray images for pneumonia detection. It must analyze color fundus images for DR classification.

- **Comparative Performance Analysis**

  The system should compare its performance with traditional machine learning methods such as KNN, SVM, Random Forest, and Naïve Bayes. It must report improvements in accuracy, processing speed, and computational efficiency.

- **User-Friendly Interface**

  The system should provide a user-friendly interface for medical professionals to input data and view diagnostic results. It must support the upload of medical images in standard formats for analysis.

- **Model Training and Testing**

  The system must train on labeled medical datasets and validate performance using independent testing sets. It should utilize performance metrics like accuracy, precision, recall, and Cohen's kappa coefficient.

- **Scalability and Adaptability**

The system should be scalable to accommodate additional diseases in the future.It must adapt to different medical imagining modalities for broade apllications

3.1.2 Non-functional Requirements

These are basically the quality constraints that the system must satisfy according to the project contract. The priority or extent to which these factors are implemented varies from one project to other. They are also called non-behavioral requirements.

- **Performance**:
  The system should ensure high detection speed and accuracy, minimizing false positives and false negatives while processing large datasets efficiently.
- **Scalability:**
  The system must be able to scale and handle increased traffic, ensuring real-time phishing detection for large volumes of websites without performance degradation.
- **Security:**
  The system must ensure secure handling of sensitive user data and prevent tampering with the ML models or detection process to maintain the integrity of the results.
- **Reliability:**
  The system should ensure consistent uptime and availability, especially during high-demand periods, to reliably detect phishing websites at all times.
- **Usability:**
  The system should have an intuitive interface for end-users, providing clear, understandable alerts and explanations for detected phishing sites without requiring deep technical knowledge.

3.2 Feasibility Study

The feasibility of the project is analysed in this phase and business proposal is put forth with a very general plan for the project and some cost estimates. During system analysis the feasibility study of the proposed system is to be carried out. This is to ensure that the proposed system is not a burden to the company. For feasibility analysis, some understanding of the major requirements for the system is essential.

Three key considerations involved in the feasibility analysis are

- Technical Feasibility
- Economical Feasibility
- Social Feasibility

3.2.1 Technical Feasibility

This study is carried out to check the technical feasibility, that is, the technical requirements of the system. Any system developed must not have a high demand on the available technical resources. This will lead to high demands on the available technical resources. This will lead to high demands being placed on the client. The developed system must have a modest requirement, as only minimal or null changes are required for implementing this system.

3.2.2 Economical Feasibility

This study is carried out to check the economic impact that the system will have on the organization. The amount of fund that the company can pour into the research and development of the system is limited. The expenditures must be justified. Thus, the developed system as well within the budget and this was achieved because most of the technologies used are freely available. Only the customized products had to be purc hased.

3.2.3 Social Feasibility

The aspect of study is to check the level of acceptance of the system by the user. This includes the process of training the user to use the system efficiently. The user must not feel threatened by the system, instead must accept it as a necessity. The level of acceptance by the users solely depends on the methods that are employed to educate the user about the system and to make him familiar with it. His level of confidence must be raised so that he is also able to make some constructive criticism, which is welcomed, as he is the final user of the system.

3.3 Proposed System Overview

The proposed system utilizes ResNet, CNN, MobileNet, and DenseNet architectures to enhance the diagnosis of heart disease, pneumonia, and diabetic retinopathy.

• **MobileNet**: Lightweight and computationally efficient, making it ideal for real-time applications and mobile devices.

• **DenseNet**: Improved feature reuse through dense connections, leading to better accuracy and convergence rates.

• **ResNet**: Deep residual learning helps in addressing vanishing gradient issues, achieving high accuracy and robust feature extraction.

• **CNN**: Acts as the foundation of the system, enabling deep feature extraction and efficient medical image analysis.

The system integrates pre-processing steps for datasets, hyperparameter optimization, and multi-disease classification with improved precision and recall metrics.

Advantages of Proposed System:

1.Higher Accuracy: Achieves superior classification performance compared to traditional methods, with ResNet contributing to the highest accuracy.

2.Computational Efficiency: Reduces computational complexity with MobileNet's lightweight architecture.

3.Scalability: DenseNet's feature reuse enables robust performance across multiple datasets.

4.Real-time Capability: Faster inference times make it suitable for clinical deployment.

5.Versatility: Unified framework capable of handling heterogeneous medical datasets.

List of Algorithms: MobileNet, DenseNet, ResNet, CNN.

Chapter 4: System Design

4.1 System Architecture

System architecture is a conceptual model that describes the structure and behavior of multiple components and subsystems.

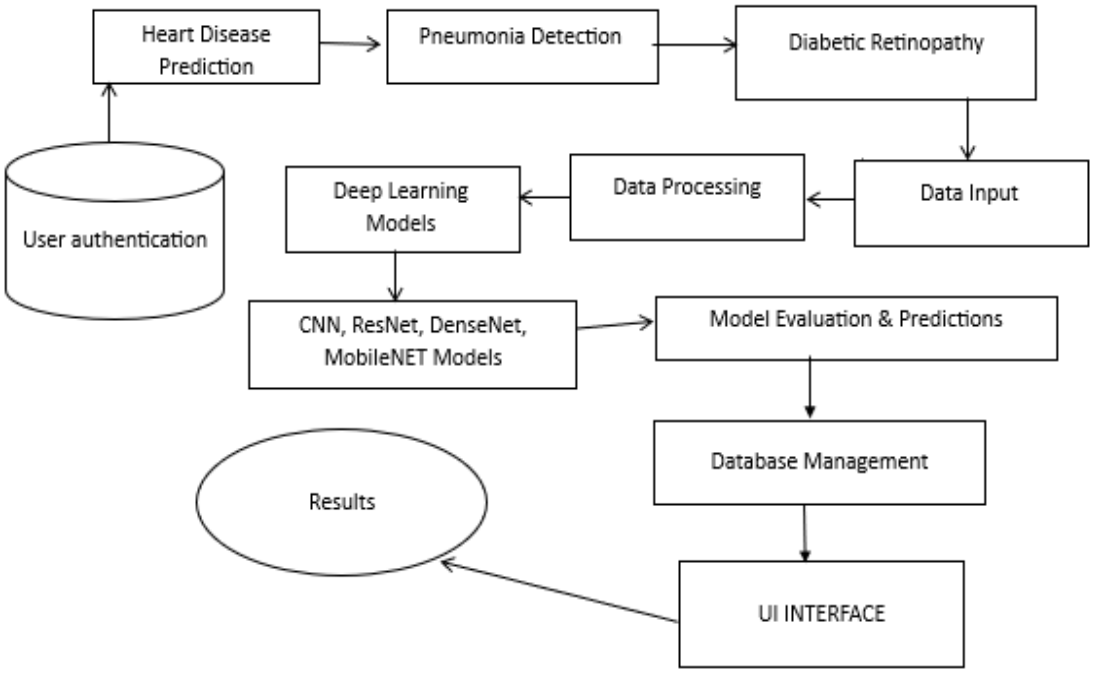

Fig 4.1.1: System Architecture Diagram

4.2 Block Diagram

A block diagram is a graphical representation of a system, project, or scenario. It provides a functional view of a system and illustrates how the different elements of that system interlink. Engineers, in particular, use block diagrams to model the elements of a system and understand how they are all connected. But these tools are also helpful in many other use cases.

Creating block diagrams helps everyone involved in a project understand and visualize what's needed for the whole system to work. By providing a clear understanding of the elements, how they're linked, and how they work together, block diagrams make it easier to keep your team on the same page and achieve the best possible system design.

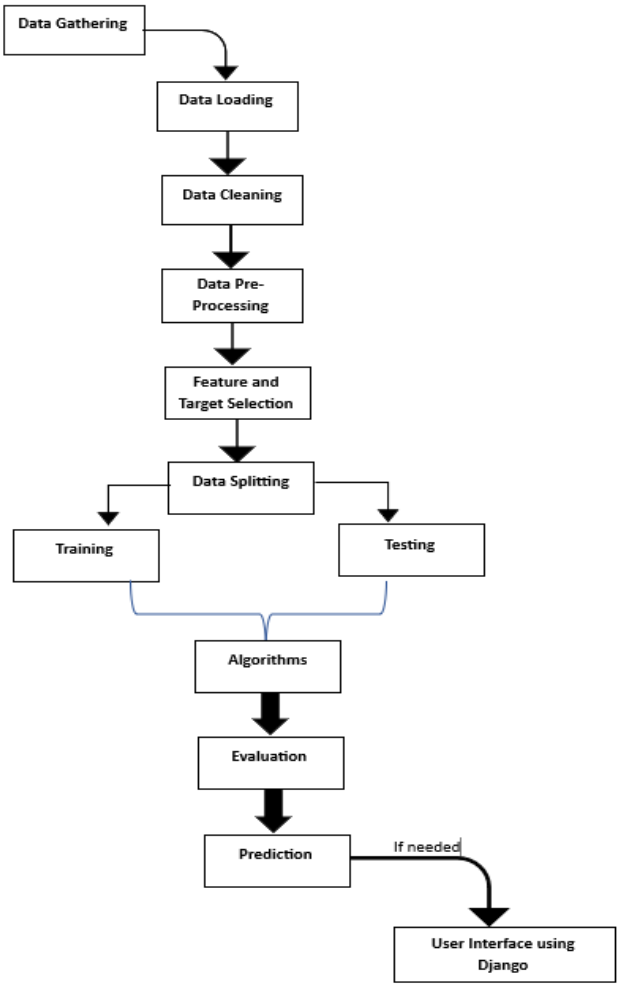

Fig 4.2.1: Block Diagram

4.3 Data Flow Diagram

Data Flow Diagram describes the processes that are involved in a system to transfer data from the input to the file storage and reports generation.

Purpose

- DFD graphically representing the functions, or processes, which capture manipulate, store, and distribute data between a system and its environment and between the components of a system.

Components

- External Entity
- Process
- Data Flow

- Data store

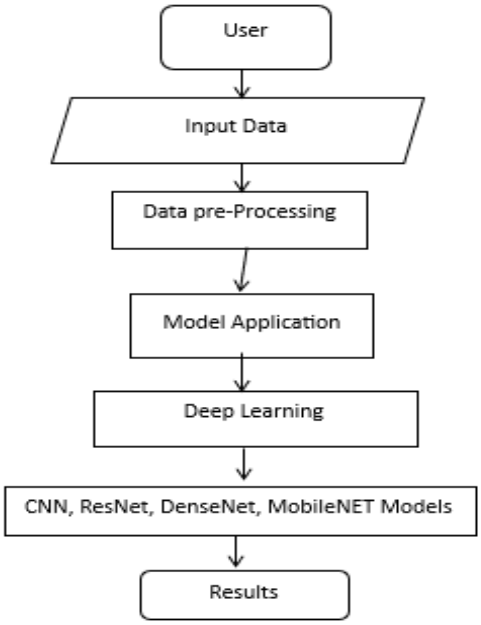

Fig 4.3.1: Data Flow Diagram

4.4 UML Diagrams

4.4.1 Use Case Diagram

A use case diagram is used to represent the dynamic behavior of a system. It encapsulates the system's functionality by incorporating use cases, actors, and their relationships.

Purpose

- The main purpose of a use case diagram is to portray the dynamic aspect of a system. It accumulates the system's requirement, which includes both internal as well as external influences.

Components

- Use Case
- Actor
- Association Or Communication Link in Boundary of System

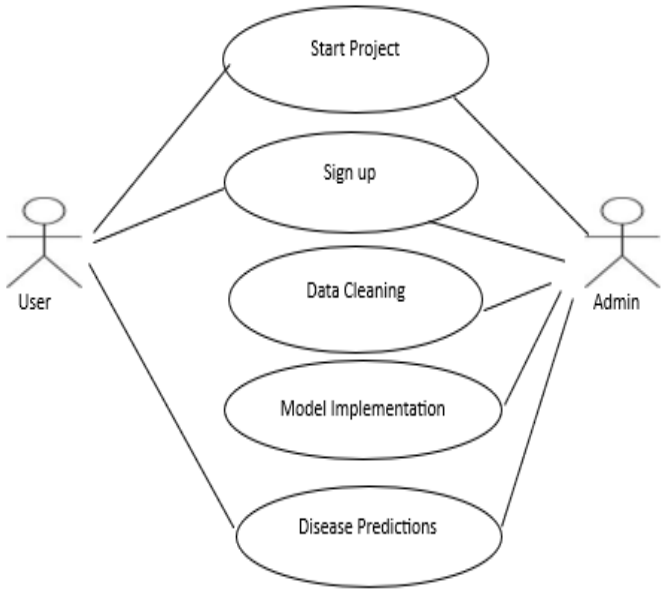

Fig 4.4.1.1: Use Case Diagram

4.4.2 Class Diagram

Class diagram is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, operations and the relationship among the objects.

Purpose

- Shows static structure of classifiers in a system.
- Helpful for developers and other team members too
- Business analysts can use class diagram to model systems from a business perspective.

Components

- Class Name
- Class Attribute
- Class Operations
- Association
- Self-Association
- Dependency

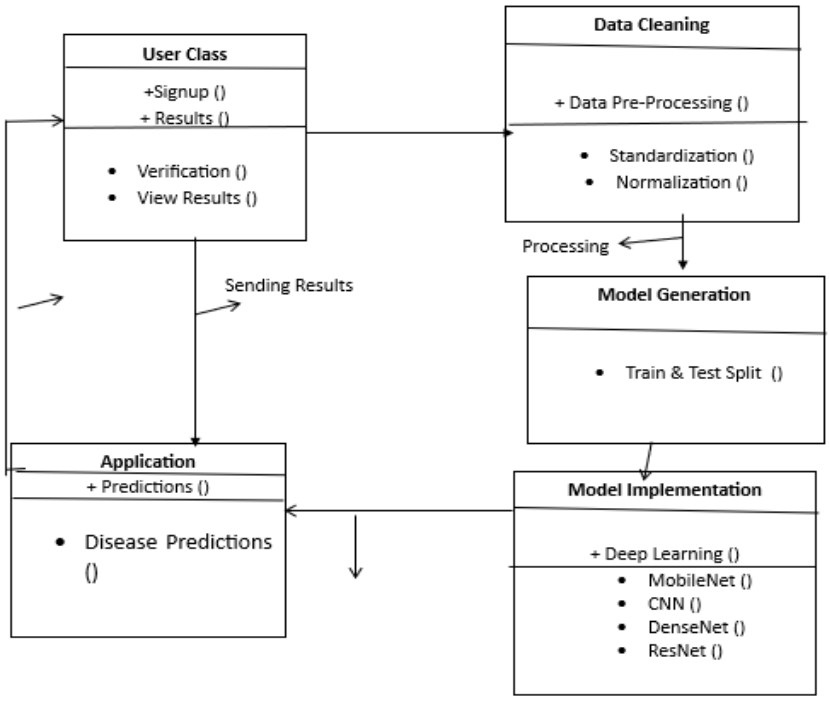

Fig 4.4.2.1: Class Diagram

4.4.3 Sequence Diagram

Sequence diagrams are interaction diagrams that detail how operations are carried out. They capture the interaction between objects in the context of collaboration.

Purpose

- Model high – level interaction between active objects in a system.
- Model the interaction between object instances with collaboration that realize a use case.

Components

- Actor
- Life Line
- Activation
- Message
- Reply Message

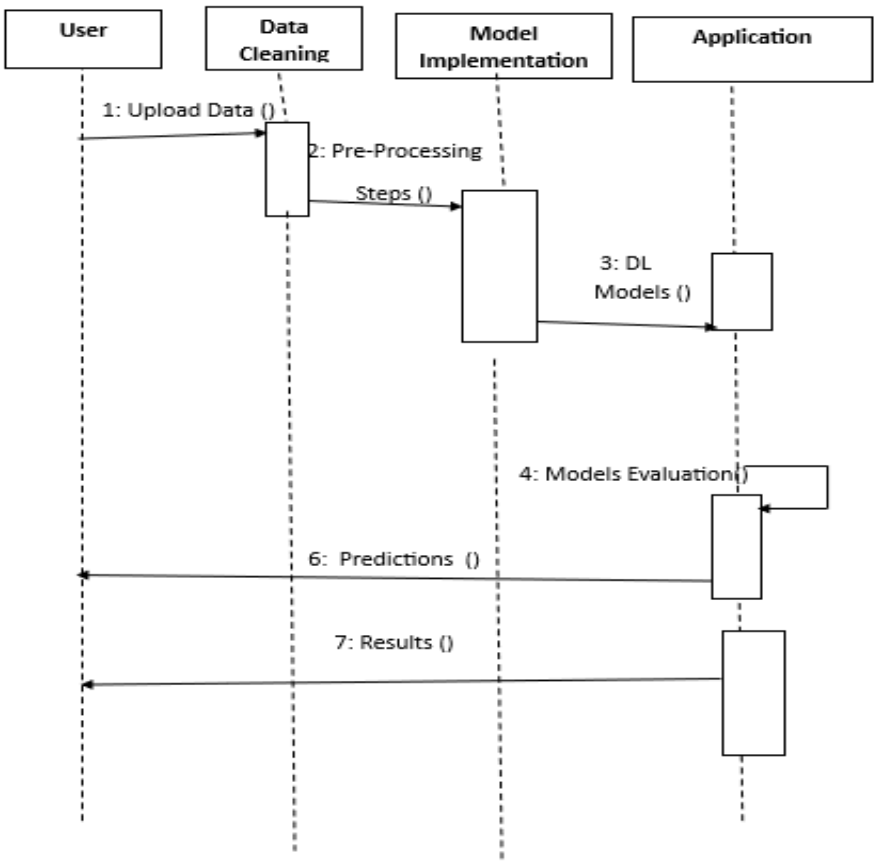

Fig 4.4.3.1: Sequence Diagram

4.4.4 Activity Diagram

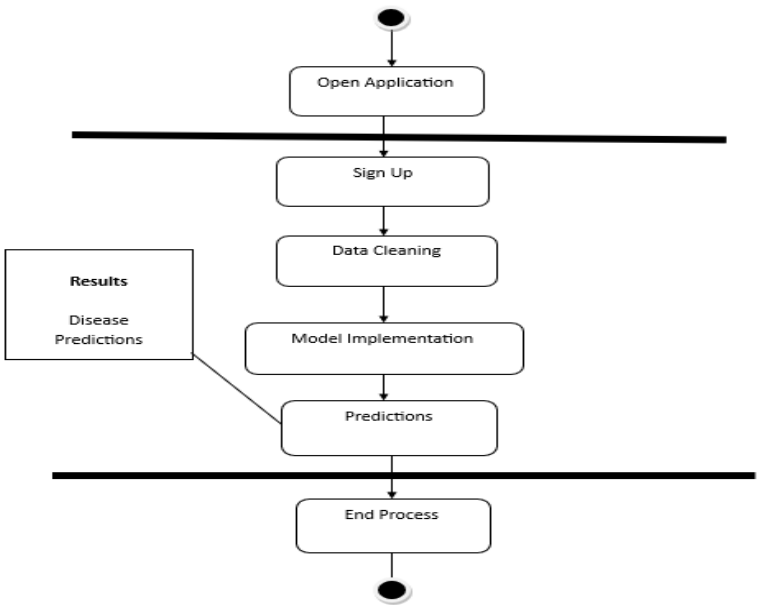

Fig 4.4.4.1: Activity Diagram

4.5 Database Design

Database design is a crucial step in the development of any software system, ensuring efficient data storage, retrieval, and management. It involves structuring the database to optimize performance, maintain integrity, and support scalability.

Tables Created for this Project:

- Admin
- Audit_log
- OTP
- Prediction
- User

```
+-------------------------------------+
| Tables_in_multi_disease_prediction |
+-------------------------------------+
| admin                               |
| audit_log                           |
| otp                                 |
| prediction                          |
| user                                |
+-------------------------------------+
5 rows in set (0.03 sec)
```

Fig 4.5.1: Tables Created for the Project

```
+------------+--------------+------+-----+-------------------+-------------------+
| Field      | Type         | Null | Key | Default           | Extra             |
+------------+--------------+------+-----+-------------------+-------------------+
| id         | int          | NO   | PRI | NULL              | auto_increment    |
| username   | varchar(100) | NO   | UNI | NULL              |                   |
| password   | varchar(255) | NO   |     | NULL              |                   |
| created_at | datetime     | YES  |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
| role       | varchar(50)  | YES  |     | admin             |                   |
+------------+--------------+------+-----+-------------------+-------------------+
5 rows in set (0.02 sec)
```

Fig 4.5.2: Structure of Admin Table

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
admin_id	int	NO	MUL	NULL	
action	varchar(255)	NO		NULL	
target_user_id	int	YES	MUL	NULL	
timestamp	datetime	YES		CURRENT_TIMESTAMP	DEFAULT_GENERATED

5 rows in set (0.00 sec)

Fig 4.5.3: Structure of Audit_log Table

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
email	varchar(100)	NO	UNI	NULL	
otp	varchar(6)	NO		NULL	
created_at	datetime	YES		CURRENT_TIMESTAMP	DEFAULT_GENERATED
expires_at	datetime	YES		NULL	

5 rows in set (0.00 sec)

Fig 4.5.4: Structure of OTP Table

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
user_id	int	NO	MUL	NULL	
prediction_type	varchar(50)	NO		NULL	
input_data	text	NO		NULL	
result	text	NO		NULL	
created_at	datetime	YES		CURRENT_TIMESTAMP	DEFAULT_GENERATED
status	varchar(20)	YES		completed	

7 rows in set (0.00 sec)

Fig 4.5.5: Structure of Prediction Table

```
+-------------------+--------------+------+-----+-------------------+-------------------+
| Field             | Type         | Null | Key | Default           | Extra             |
+-------------------+--------------+------+-----+-------------------+-------------------+
| id                | int          | NO   | PRI | NULL              | auto_increment    |
| name              | varchar(100) | NO   |     | NULL              |                   |
| email             | varchar(100) | NO   | UNI | NULL              |                   |
| password          | varchar(255) | NO   |     | NULL              |                   |
| mobile            | varchar(20)  | NO   |     | NULL              |                   |
| dob               | date         | NO   |     | NULL              |                   |
| security_question | varchar(255) | NO   |     | NULL              |                   |
| security_answer   | varchar(255) | NO   |     | NULL              |                   |
| status            | varchar(20)  | YES  |     | inactive          |                   |
| created_at        | datetime     | YES  |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
+-------------------+--------------+------+-----+-------------------+-------------------+
10 rows in set (0.00 sec)
```

Fig 4.5.6: Structure of User Table

Chapter 5: Implementation

5.1 Programming Languages and Technologies Used

- **Python:** Python is the primary programming language used for developing the system due to its versatility, extensive libraries, and support for machine learning and web development frameworks.
    - Python's simple syntax makes it easy to develop complex systems efficiently.
    - Its large community ensures constant updates, support, and availability of numerous third-party libraries.
- **TensorFlow and Keras:** TensorFlow and Keras are popular deep learning frameworks in Python used for building and training Convolutional Neural Network (CNN) models for image analysis tasks.
    - TensorFlow provides a flexible ecosystem for developing end-to-end machine learning workflows.
    - Keras, as a high-level API, simplifies model building and debugging, making it ideal for rapid prototyping.
- **Scikit-learn:** Scikit-learn is a machine learning library in Python used for implementing various machine learning algorithms, including preprocessing techniques and model evaluation.
    - It offers tools for splitting datasets, feature scaling, and hyperparameter tuning.
    - Its robust implementation ensures compatibility with other Python libraries like Pandas and NumPy.
- **Pandas and NumPy:** Pandas and NumPy are Python libraries used for data manipulation and numerical computations, respectively. They are essential for handling and processing the dataset of skin images.
    - Pandas provides powerful data structures for analyzing and manipulating structured data.
    - NumPy supports efficient mathematical operations and handling of large arrays or matrices.

- **Matplotlib and Seaborn:** Matplotlib and Seaborn are Python libraries used for data visualization, facilitating the visualization of skin image features and model evaluation metrics.
    - Matplotlib allows the creation of detailed plots and graphs for data insights.
    - Seaborn builds on Matplotlib by offering more visually appealing and statistically enriched plots.
- **Flask:**
    - Flask is a lightweight and flexible web framework in Python, designed for building web applications with minimal overhead. It follows a micro-framework approach, providing essential tools for request handling, routing, and templating without imposing strict project structures.
    - Flask's simplicity makes it ideal for small to medium-sized applications, while its extensibility allows integration with various third-party libraries.
    - Unlike Django, Flask does not include an in-built ORM but supports SQL Alchemy for efficient database management. Its minimalistic design and modular nature make it a preferred choice for developers seeking greater control over their application architecture.
- **Anaconda:** Anaconda is a distribution of Python that includes pre-installed packages and environments for data science and machine learning tasks. It simplifies the setup and management of the development environment.
    - Anaconda's package manager, Conda, helps manage dependencies and ensures compatibility across libraries.
    - It also comes with Jupyter Notebook, a tool widely used for exploratory data analysis and visualization.
- **HTML, CSS, and JavaScript**
    - Description: HTML (HyperText Markup Language) provides the structure, CSS (Cascading Style Sheets) defines the design, and JavaScript enables dynamic behavior in web pages.
    - Usage: HTML is used for structuring web pages, CSS for styling the user interface, and JavaScript for interactive features like dynamic plant searches, filtering.

Code:

```
import numpy as np

import tensorflow as tf

import tensorflow as tf

import os

import matplotlib.pyplot as plt

import cv2

import numpy as np

from sklearn.model_selection import train_test_split

import warnings

warnings.filterwarnings("ignore")


# Notes

num_samples = 100

image_shape = (64, 64, 3)

quantum_data_shape = (64,)


# Load and preprocess data from subfolders

def load_data(folder):

    images = []

    labels = []

    class_names = sorted(os.listdir(folder))

    for class_index, class_name in enumerate(class_names):

        class_path = os.path.join(folder, class_name)
```

```
        for image_name in os.listdir(class_path):
            image_path = os.path.join(class_path, image_name)
            image = cv2.imread(image_path)
            image = cv2.resize(image, (64,64))
            images.append(image)
            labels.append(class_index)
    return images, labels

# Load and preprocess image data
images, labels = load_data('Retinopath_images\data')

# Split data into train and test sets
train_images, test_images, train_labels, test_labels = train_test_split(
    images, labels, test_size=0.2, random_state=42, stratify=labels
)
# Convert lists to numpy arrays
train_images = np.array(train_images)
train_labels = np.array(train_labels)
test_images = np.array(test_images)
test_labels = np.array(test_labels)
# Normalize pixel values to the range [0, 1]
train_images = train_images.astype('float32') / 255.0
test_images = test_images.astype('float32') / 255.0
```

```
# Print shapes for verification
print("Train images shape:", train_images.shape)
print("Train labels shape:", train_labels.shape)
print("Test images shape:", test_images.shape)
print("Test labels shape:", test_labels.shape)
import numpy as np
from qiskit import QuantumCircuit, Aer, transpile, assemble
def create_quantum_circuit(image_features):
    num_features = len(image_features)
    quantum_register = QuantumCircuit(num_features, num_features)
    for i in range(num_features):
        value = np.mean(image_features[i])
        scaled_value = value * np.pi / 255.0
        quantum_register.ry(scaled_value, i)
    quantum_register.measure_all()
    return quantum_register
import numpy as np
from qiskit import QuantumCircuit, Aer, transpile, assemble
def qsdl(set1):
    train_q = set1
    num_images = len(train_q)
    num_qubits = 6
    num_outcomes = 2 ** num_qubits
    quantum_data = np.zeros((num_images, num_outcomes), dtype=int)
```

```
    simulator = Aer.get_backend('aer_simulator')
    for i, image_features in enumerate(train_q):
        qc = create_quantum_circuit(image_features)
        compiled_circuit = transpile(qc, simulator)
        qobj = assemble(compiled_circuit)
        result = simulator.run(qobj).result()
        counts = result.get_counts()
        for outcome, count in counts.items():
            original_string = outcome
            without_spaces = "".join(original_string.split())
            outcome_int = int(without_spaces, 2)
            if outcome_int < num_outcomes:
                quantum_data[i, outcome_int] = count
    print("Quantum data created")
    return quantum_data
cnn_input = tf.keras.layers.Input(shape=image_shape)
cnn_layer = tf.keras.layers.Conv2D(filters=32, kernel_size=(3, 3),
activation='relu')(cnn_input)
cnn_flatten = tf.keras.layers.Flatten()(cnn_layer)

# Define the quantum data input
quantum_input = tf.keras.layers.Input(shape=quantum_data_shape)

# Combine CNN and quantum data
combined = tf.keras.layers.Concatenate()([cnn_flatten, quantum_input])
```

```
# Fully connected layers
dense1 = tf.keras.layers.Dense(128, activation='relu')(combined)
output = tf.keras.layers.Dense(5, activation='softmax')(dense1)

# Create the model
model = tf.keras.models.Model(inputs=[cnn_input, quantum_input], outputs=output)

# Compile the model
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy',
metrics=['accuracy'])

# Print the model summary
model.summary()
quantum_data=qsdl(train_images[:])
print("quantum data created for training images")
history=model.fit([train_images[:], quantum_data[:]], train_labels[:], epochs=60,
batch_size=32)
info=history.history
info
plt.figure(figsize=(5,5))
plt.plot(info['loss'],color="red",label="loss")
plt.show()
 plt.figure(figsize=(5,5))
plt.plot(info['accuracy'],color="green",label="loss")
```

```
plt.show()
quantum_data_test=qsdl(test_images[:])
from sklearn.metrics import precision_score, recall_score, f1_score, accuracy_score
predictions = model.predict([test_images[:], quantum_data_test[:]])
predicted_labels = np.argmax(predictions, axis=1)
print(predicted_labels)
Actual={0:"Mild",1:"Moderate",2:"No_DR",3:"Proliferate_DR",4:"Severe"}
for i in predicted_labels:
   print("The class of the image is -->",Actual[i])
precision = precision_score(test_labels[:], predicted_labels, average='weighted')
recall = recall_score(test_labels[:], predicted_labels, average='weighted')
f1 = f1_score(test_labels[:], predicted_labels, average='weighted')
accuracy = accuracy_score(test_labels[:], predicted_labels)
print(f"Precision:{precision*100:.2f}%")
print(f"Recall:{ recall*100:.2f}%")
print(f"F1 Score:{f1*100:.2f}%")
print(f"Accuracy: {accuracy*100:.2f}%")
images_list=[]
prediction_list=[]
from numpy import *
def pred(name):
    unseen=[]
    display_image=plt.imread(name)
    image=cv2.imread(name)
```

```
    image=cv2.resize(image, (64,64))
    unseen.append(image)
    unseen_images = np.array(unseen)
   unseen_images = unseen_images.astype('float32') / 255.0
   quantum_unseen=qsdl(unseen_images[:1])
   predictions = model.predict([unseen_images[:1],    quantum_unseen[:1]])
  print("Predicted probabilities:")
  predict_label = np.argmax(predictions, axis=1)
  max_predicted_label = np.max(predict_label)
  print("Predicted labels:", predict_label)
Actual={0:"Mild",1:"Moderate",2:"No_DR",3:"Proliferate_DR",4:"Severe"}
   print("The class of the images is-->", Actual[max_predicted_label])
   images_list.append(display_image)
   prediction_list.append(predict_label)
for i in range(1,7):
pred('p'+str(i)+'.png')
fig,ax=plt.subplots(nrows=3,ncols=2,figsize=(20,20))
ax=ax.flatten()
for i,j in enumerate(images_list[:6]):
   ax[i].imshow(j) #already in minimal scaled range
   ax[i].title.set_text("Label: "+str(prediction_list[i][0])+"; The class of the image is "+str(Actual[prediction_list[i][0]]))
model.save('diabetic.keras')
```

5.2 Development Tools and Environment

- **Anaconda Distribution:**

- The Anaconda Distribution is utilized as a comprehensive platform for Python development and data science tasks.
- It provides easy access to a wide range of Python libraries and packages essential for fabric defect detection, including NumPy, Pandas, TensorFlow, and Matplotlib.
- Conda, the package manager included in Anaconda, simplifies the management of libraries, dependencies, and virtual environments, enhancing reproducibility and project organization.

- **Jupyter Notebook:**
  - Jupyter Notebook serves as an interactive computing environment for prototyping, experimentation, and documentation of fabric defect detection workflows.
  - It enables seamless integration of code, visualizations, and narrative text, facilitating data exploration, model development, and result analysis.
  - The notebook format allows for the creation of comprehensive and reproducible reports, capturing the entire fabric defect detection process from data preprocessing to model evaluation.
- **TensorFlow:**
  - TensorFlow, developed by Google, is a fundamental deep learning framework utilized for building, training, and deploying deep neural networks, including ResNet architectures.
  - It provides a scalable and efficient platform for implementing complex deep learning models tailored to fabric defect detection tasks.
  - TensorFlow's high-level APIs and extensive tooling support enable efficient experimentation, optimization, and deployment of fabric defect detection models.
- **Keras:**
  - Keras serves as a user-friendly and modular neural networks API, often used as a frontend for TensorFlow.
  - It simplifies the process of defining, configuring, and training deep learning models, including ResNet variants, for fabric defect detection.

- Keras's intuitive interface and abstraction layers streamline model development, making it accessible to researchers and practitioners with varying levels of expertise.

- **OpenCV:**
  - OpenCV (Open-source Computer Vision Library) is a powerful tool for image processing and computer vision tasks.
  - It provides functions for image preprocessing, feature extraction, and object detection, which are essential for fabric defect detection.
  - OpenCV's extensive collection of algorithms and utilities enables developers to perform various image analysis tasks efficiently.
- **Flask**
  - Description: Flask is a lightweight Python web framework that is used for building web applications. It provides essential features such as routing, request handling, and templating without the overhead of larger frameworks.
  - Usage: Manages the core application logic, handles different routes for plant information, herbal teas, plant identification, and user feedback. It integrates with MySQL via SQLAlchemy and uses Jinja2 for rendering dynamic web pages.
- **MySQL**
  - Description: MySQL is an open-source relational database management system.
  - Usage: Stores data such as plant information, medicinal remedies, yoga poses, herbal tea recipes, user feedback, and more. It is connected to Flask using SQLAlchemy for data retrieval and storage.

5.3 Module-wise Implementation Details

5.3.1 Data Acquisition and Preprocessing Module

The module exists to procure medical images in proper form for model training and testing activities. The analysis relies on the combination of UCI Heart Disease

dataset with chest X-ray images and color fundus images. Medical image quality enhancement happens through preprocessing which includes noise reduction and contrast enhancement to achieve consistency within processed images. The goal of this module is to obtain medical data which requires cleaning before it can be used as input for deep learning model testing. The data requires optimization before it moves to the next processing modules.

5.3.2 Model Development Module

The implementation includes a creation process for deep learning models that feature CNN alongside ResNet and MobileNet and DenseNet structures. The detection of pneumonia occurs through usage of the VGG16 network architecture. The model performance along with efficiency benefits from multiple hyperparameter optimization approaches. The main goal of this project is to create steady deep learning algorithms that effectively identify heart disease together with pneumonia and diabetic retinopathy. Improving diagnostic performance requires proper optimization of the selected network architectures.

5.3.3 Training and Evaluation Module

The pretrained deep learning models receive their training inputs from preprocessed datasets. The system uses accuracy, precision, recall along with Cohen's kappa coefficient to evaluate model performance. The research implements a benchmark test between deep learning models and traditional machine learning frameworks which contain KNN, SVM, Random Forest and Naïve Bayes. The models need medical data to undergo training for accuracy assessment during classification. A research objective exists to prove deep learning techniques surpass conventional machine learning approaches.

5.3.4 User Interface Module

The system features an interface that provides users with an easy way to interact with it. Through the interface medical staff have the ability to enter patient information combined with medical image uploads for assessment purposes. The system presents diagnostic outputs in easily understandable methods. The design offers an intuitive platform which allows users to access the diagnostic system assessment capabilities. The user interface exists to provide convenient access to all healthcare professionals.

5.3.5 Deployment and Maintenance Module

The implementation module delivers necessary deployment capabilities for the developed system that practitioners can use in clinical practice. The deployment module requires continuous support through updates, fixes, and maintenance activities which strengthen system stability along with resolving latest problems. The deployment module supports the implementation process of the diagnostic system within medical facilities. The system will receive ongoing maintenance through this module for continuous development along with long-term use. The systematic process guarantees automation between data acquisition and system deployment operations resulting in enhanced diagnostic system efficiency.

5.4 Algorithms and Logic Used

5.4.1 MobileNet

MobileNet is a lightweight deep convolutional neural network optimized for mobile and edge devices, designed using depth wise separable convolutions to reduce computational cost while maintaining high accuracy. It is widely used for real-time applications such as object detection, facial recognition, and medical image analysis on resource-constrained devices. MobileNet assumes that reducing the number of parameters and computations will not significantly affect model performance. Its advantages include low latency, reduced memory footprint, and efficient performance on mobile and embedded systems. However, its limitations include slightly lower accuracy compared to larger architectures and potential difficulties in handling highly complex patterns due to its compact design.

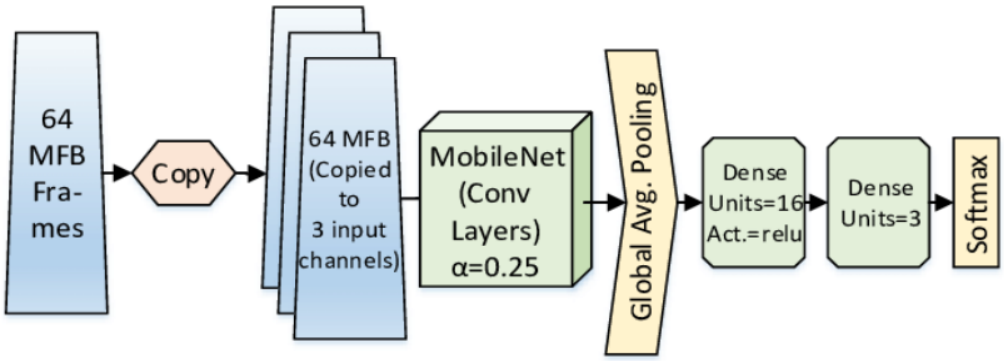

Fig 5.4.1.1: MobileNet Architecture

5.4.2 DenseNet

DenseNet (Densely Connected Convolutional Networks) enhances feature reuse by connecting each layer to every other layer, allowing for efficient gradient flow and improved learning. It is commonly used in medical image segmentation, classification tasks, and object recognition. The key assumption of DenseNet is that densely connected layers improve feature propagation and reduce redundancy. Its advantages include improved accuracy with fewer parameters, reduced vanishing gradient issues, and enhanced feature reuse. However, DenseNet has high memory requirements due to its dense connections and can be computationally intensive, making it challenging for deployment on memory-constrained devices.

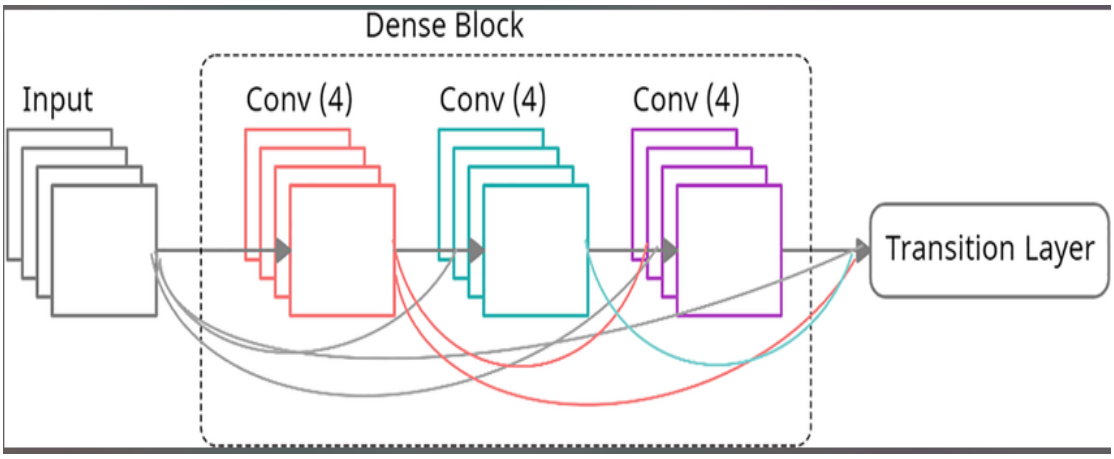

Fig 5.4.2.1: DenseNet Block

The Dense Block belongs to the DenseNet framework that features a dense network topology for all its architecture. The Dense Block enables all preceding

convolutional layers to send their input to the current layer as well as enable its output to pass through to all subsequent layers. The Conv(4) operations generate four feature maps across each layer while complete output sequences from all prior layers become concatenated in every succeeding layer which enhances both parameter sharing and backward error propagation. A Transition Layer follows the dense block to both lower the map count and decrease spatial dimensions thereby managing model complexity. The model structure enables better efficiency and reduces difficulties with gradient vanishing through its design.

5.4.3 ResNet

ResNet (Residual Neural Network) introduces skip (residual) connections to address the vanishing gradient problem, enabling the training of very deep networks. It is widely used in medical imaging, autonomous driving, and various computer vision tasks requiring deep feature extraction. The fundamental assumption of ResNet is that learning identity mappings through residual connections helps optimize deep architectures. Its advantages include improved convergence, higher accuracy in deep models, and effective training of very deep networks. However, ResNet can be computationally expensive, and deeper versions (e.g., ResNet-152) may still suffer from redundancy, requiring careful parameter tuning for optimal performance.

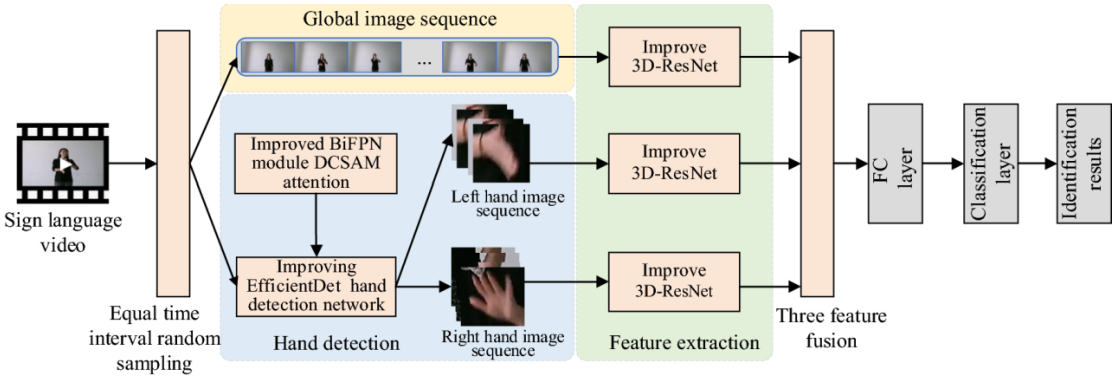

Fig 5.4.3.1: ResNet Architecture

5.4.4 CNN

CNN (Convolutional Neural Networks) is a deep learning architecture designed for spatial data processing, particularly image and video analysis. It consists of convolutional layers, pooling layers, and fully connected layers to extract hierarchical features. Use cases include facial recognition, self-driving cars, and medical diagnosis.

Assumptions include the presence of spatial hierarchies in data, benefiting from localized feature extraction. Advantages include automated feature learning, translational invariance, and high accuracy in visual tasks. Limitations involve high computational requirements, dependency on large labeled datasets, and vulnerability to adversarial attacks.

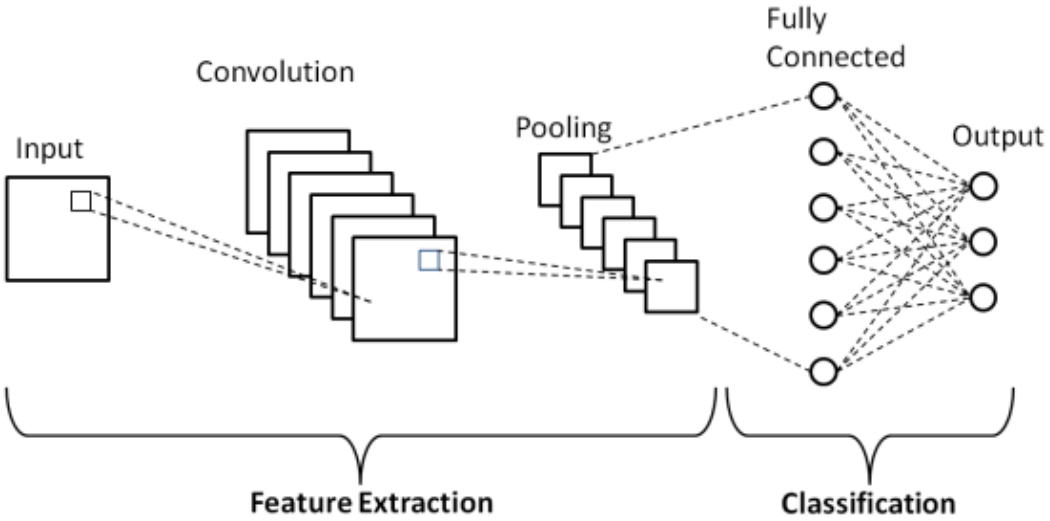

Fig 5.4.4.1: CNN Architecture

Architecture Overview

Input Layer

- Accepts images of shape (124, 124, 3).

Convolution Blocks (Feature extraction)

- Three blocks with 2 Conv layers each:
  - Block 1: 2×Conv2D(64)
  - Block 2: 2×Conv2D(128)
  - Block 3: 2×Conv2D(256)
- All followed by BatchNormalization and MaxPooling.

Flatten + Dense Layers (Classification head)

- Flatten to convert 3D to 1D.
- Dense(128) + ReLU + BatchNorm + Dropout(0.5) → regularizes model.

- Dense(6) + Softmax → for 6-class output.

Compilation

- Loss: categorical_crossentropy (since this is multi-class classification)
- Optimizer: adam
- Metric: accuracy

Chapter 6: Testing and Results

6.1 Testing Methodologies

Software testing methodologies define structured approaches to ensure the quality, reliability, and security of an application.

6.1.1 Unit Testing

Unit testing functions as a fundamental software testing approach for assessing individual component parts such as functions or methods independently to validate their correct performance. The testing approach finds defects before software deployment while enabling developer code optimization and resulting in better software excellence. Developers employ frameworks namely JUnit for Java or pytest for Python to author unit tests. The documentation value of unit tests enables safe code changes and they play a crucial role in CI/CD pipelines. Unit testing achieves reliable maintainable software through its approach of isolating units and testing dependencies using mocks or stubs. Software applications developed with this foundation become robust and scalable despite not detecting integration issues.

Example:

1. Testing Flask API endpoints (e.g., /upload_Ecg_image, /get_disease_details).
2. Validating the image processing function for plant identification.

6.1.2 Integration Testing

The software testing phase of integration testing combines different group components for testing their joint interactions. The main distinction between unit testing and integration testing comes from the fact that integration testing maintains data connectivity between modules while unit testing operates on separate components. The testing procedure discovers interface problems together with communication failures and data compatibility issues at an early stage. The three primary integration techniques consist of top-down and bottom-up together with hybrid methods. Multi-component systems require integration testing because this approach verifies that individual units work in harmony as a combined system. The third phase uses JUnit

with integration suites and Postman for API testing as well as Selenium for UI interaction.

Example:

1. Testing interactions between the frontend and Flask API.
2. Verifying image upload, processing, and storage.

6.1.3 System Testing

The testing of an entire integrated and complete software system at its highest level occurs during system testing to verify all defined requirements' fulfillment. Defining conditions which verify the system meets its functional along with non-functional specifications including performance capabilities and security and usability and reliability levels. System testing takes place in simulated production conditions to check complete software functionality through realistic scenario testing. The test operations occur after finishing integration testing and proceed towards the user acceptance testing phase (UAT). Almost all projects use testing tools composed of Selenium, JMeter and Test Complete. It is vital to perform this phase because it reveals bugs that emerge exclusively when the fully operational system is running.

Example:

1. Ensuring plant identification works correctly for different image inputs.

6.2 Testing Results

Test No	Test Cases	Expected Output	Actual Output	Pass/Fail
1	Importing the Libraries	Imported	Imported without errors	Pass
2	Collecting the images data from open source and load the data	Collected and load the data	Got Access to the data and view the data	Pass
3	Data Analysis	Analysis of data	Data Analysis without errors	Pass
4	Image Pre-processing	Image Pre-processing	Image Pre-processing of data without errors	Pass
5	Feature selection & Feature Extraction	Selecting and extracting the features	Feature extraction & selection is done	Pass
6	Train & Test Split	Split the data into 70% training and 30 testing	Splitting of data is done	Pass
7	ResNet, MobileNet DenseNet, CNN Model Application	ResNet, MobileNet, DenseNet CNN model evaluated	ResNet, MobileNet, DenseNet, CNN evaluated successfully	Pass
8	Predict the results	Input the image and predict the result	Result predicted successfully	Pass

Table 6.2.1: Testing Report

6.3 Performance Evaluation

Pneumonia Prediction

Model Evaluation:

```
from sklearn.metrics import confusion_matrix, classification_report

# Compute predictions for the ResNet model
resnet_Y_pred = model.predict(test_generator)
resnet_y_pred = np.argmax(resnet_Y_pred, axis=1)
resnet_target_names = classes  # Assuming you have defined 'classes'

# Confusion matrix
resnet_cm = confusion_matrix(test_generator.classes, resnet_y_pred)
plot_confusion_matrix(resnet_cm, resnet_target_names, normalize=False, title='ResNet Confusion Matrix')

# Classification Report
print('ResNet Classification Report')
print(classification_report(test_generator.classes, resnet_y_pred, target_names=resnet_target_names))
```

```
ResNet Classification Report
              precision    recall  f1-score   support

      NORMAL       0.93      0.96      0.95       133
   PNEUMONIA       0.99      0.98      0.98       389

    accuracy                           0.97       522
   macro avg       0.96      0.97      0.97       522
weighted avg       0.97      0.97      0.97       522
```

Fig 6.3.1: Classification Report for Pneumonia Prediction

Confusion Matrix

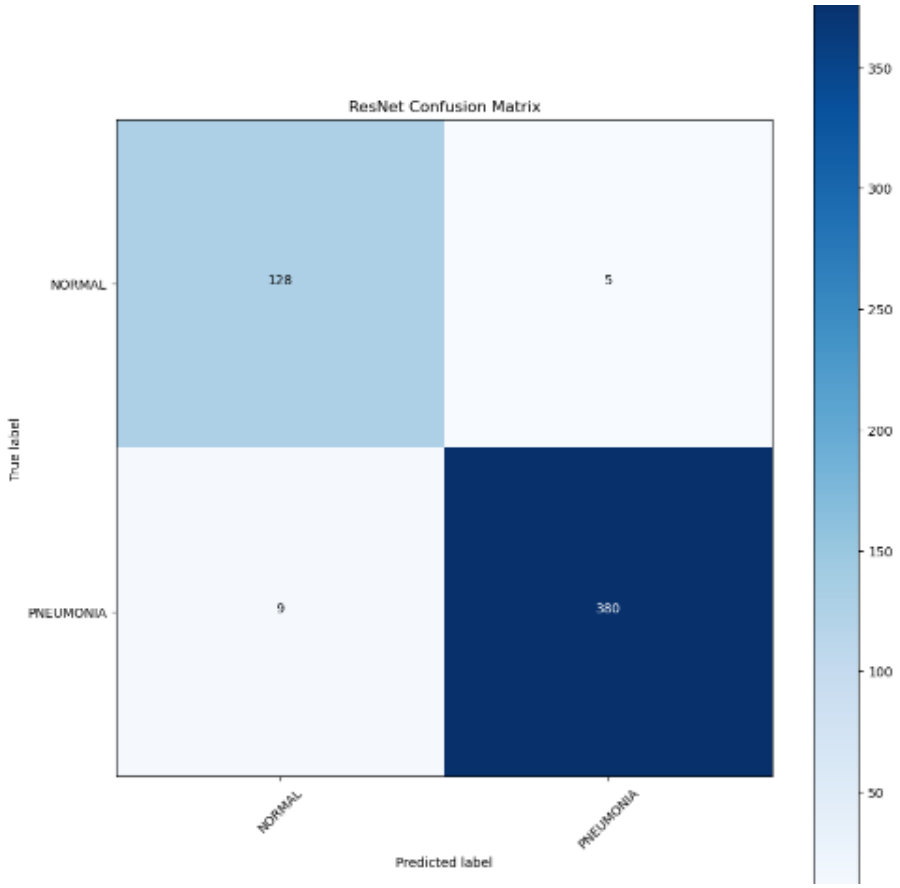

Fig 6.3.2: Confusion Matrix for Pneumonia Prediction

Heart Disease Prediction

Model Evaluation:

+ Code + Markdown

```
from sklearn.metrics import confusion_matrix, classification_report

# Compute predictions for the ResNet model
resnet_Y_pred = model.predict(test_generator)
resnet_y_pred = np.argmax(resnet_Y_pred, axis=1)
resnet_target_names = classes  # Assuming you have defined 'classes'

# Confusion matrix
resnet_cm = confusion_matrix(test_generator.classes, resnet_y_pred)
plot_confusion_matrix(resnet_cm, resnet_target_names, normalize=False, title='ResNet Confusion Matrix')

# Classification Report
print('ResNet Classification Report')
print(classification_report(test_generator.classes, resnet_y_pred, target_names=resnet_target_names))
```

```
ResNet Classification Report
              precision    recall  f1-score   support

           F       1.00      1.00      1.00        55
           M       1.00      1.00      1.00       832
           N       1.00      1.00      1.00       792
           Q       1.00      1.00      1.00       602
           S       1.00      1.00      1.00       240
           V       1.00      1.00      1.00       586

    accuracy                           1.00      3107
   macro avg       1.00      1.00      1.00      3107
weighted avg       1.00      1.00      1.00      3107
```

Fig 6.3.3: Classification Report for Heart Disease Prediction

Confusion Matrix

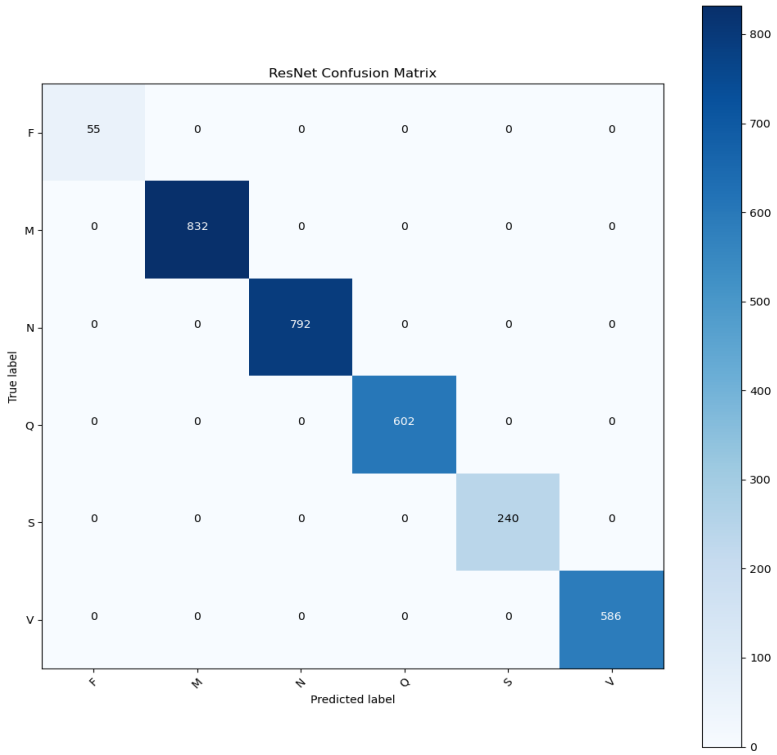

Fig 6.3.4: Confusion Matrix for Heart Disease Prediction

Diabetic Retinopathy Prediction

metrics evalution over predictions made by test data

```
from sklearn.metrics import precision_score, recall_score, f1_score, accuracy_score


predictions = model.predict([test_images[:], quantum_data_test[:]])

predicted_labels = np.argmax(predictions, axis=1)

print(predicted_labels)


Actual={0:"Mild",1:"Moderate",2:"No_DR",3:"Proliferate_DR",4:"Severe"}

for i in predicted_labels:
    print("The class of the image is -->",Actual[i])


precision = precision_score(test_labels[:], predicted_labels, average='weighted')
recall = recall_score(test_labels[:], predicted_labels, average='weighted')
f1 = f1_score(test_labels[:], predicted_labels, average='weighted')
accuracy = accuracy_score(test_labels[:], predicted_labels)


print(f"Precision:{precision*100:.2f}%")
print(f"Recall:{ recall*100:.2f}%")
print(f"F1 Score:{f1*100:.2f}%")
print(f"Accuracy: {accuracy*100:.2f}%")
```

```
The class of the image is --> Mild
The class of the image is --> No_DR
The class of the image is --> Moderate
The class of the image is --> Mild
The class of the image is --> Proliferate_DR
The class of the image is --> Proliferate_DR
The class of the image is --> Moderate
The class of the image is --> Mild
The class of the image is --> Proliferate_DR
The class of the image is --> Mild
The class of the image is --> Mild
The class of the image is --> Severe
The class of the image is --> No_DR
The class of the image is --> Proliferate_DR
The class of the image is --> Severe
The class of the image is --> Mild
The class of the image is --> Proliferate_DR
The class of the image is --> No_DR
The class of the image is --> No_DR
The class of the image is --> Proliferate_DR
The class of the image is --> Proliferate_DR
The class of the image is --> Proliferate_DR
The class of the image is --> Severe
...
Precision:86.37%
Recall:85.11%
F1 Score:84.14%
Accuracy: 85.11%
```

Fig 6.3.5: Classification Report for DR Prediction

Diabetes Prediction

```
# Classification Model
print("\033[1m Classification Report: \033[0m \n")
target_names = ['No', 'Yes']
print(classification_report(y_test, y_pred))
```

```
Classification Report:

              precision    recall  f1-score   support

           0       0.89      0.85      0.87        46
           1       0.79      0.84      0.81        31

    accuracy                           0.84        77
   macro avg       0.84      0.84      0.84        77
weighted avg       0.85      0.84      0.84        77
```

Fig 6.3.6: Classification Report for Diabetes Prediction

Confusion Matrix

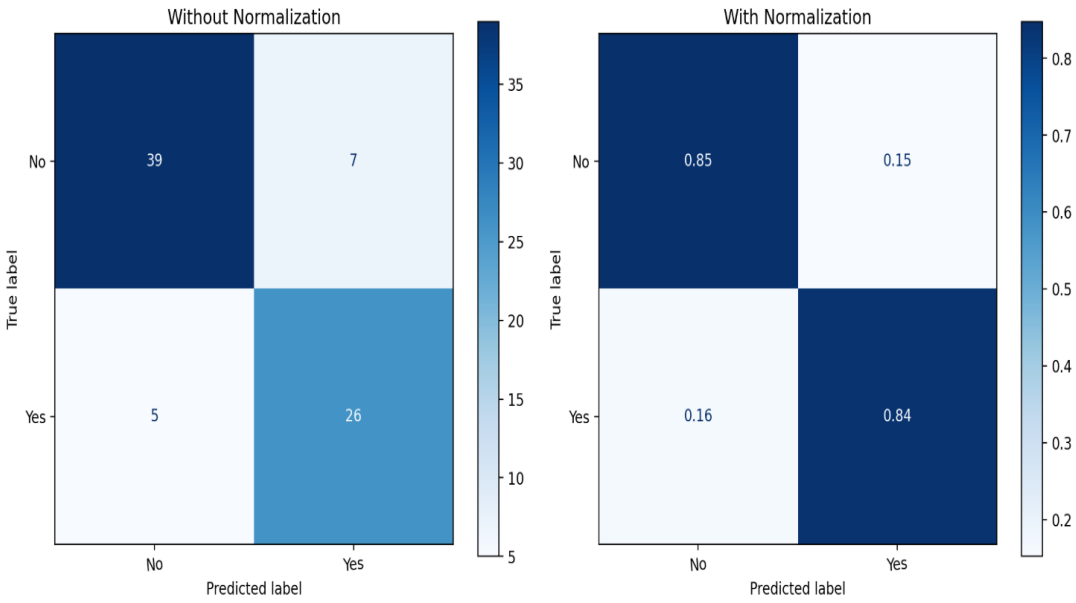

Fig 6.3.7: Confusion Matrix for Diabetes Prediction

6.4 Screenshots of Application Output

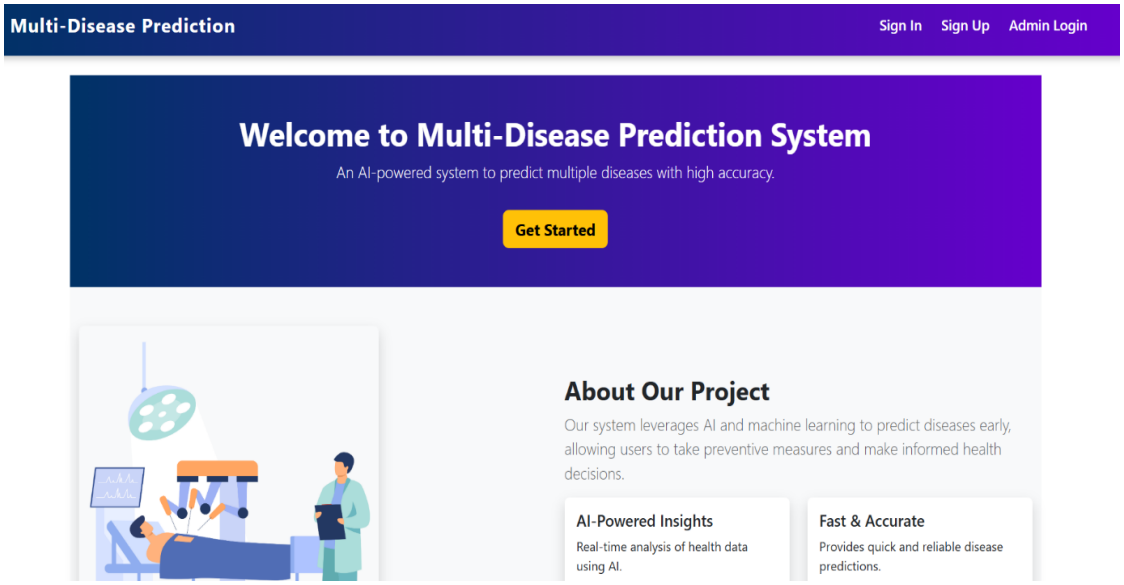

Fig 6.4.1: Homepage

Fig 6.4.2: Services of the Project

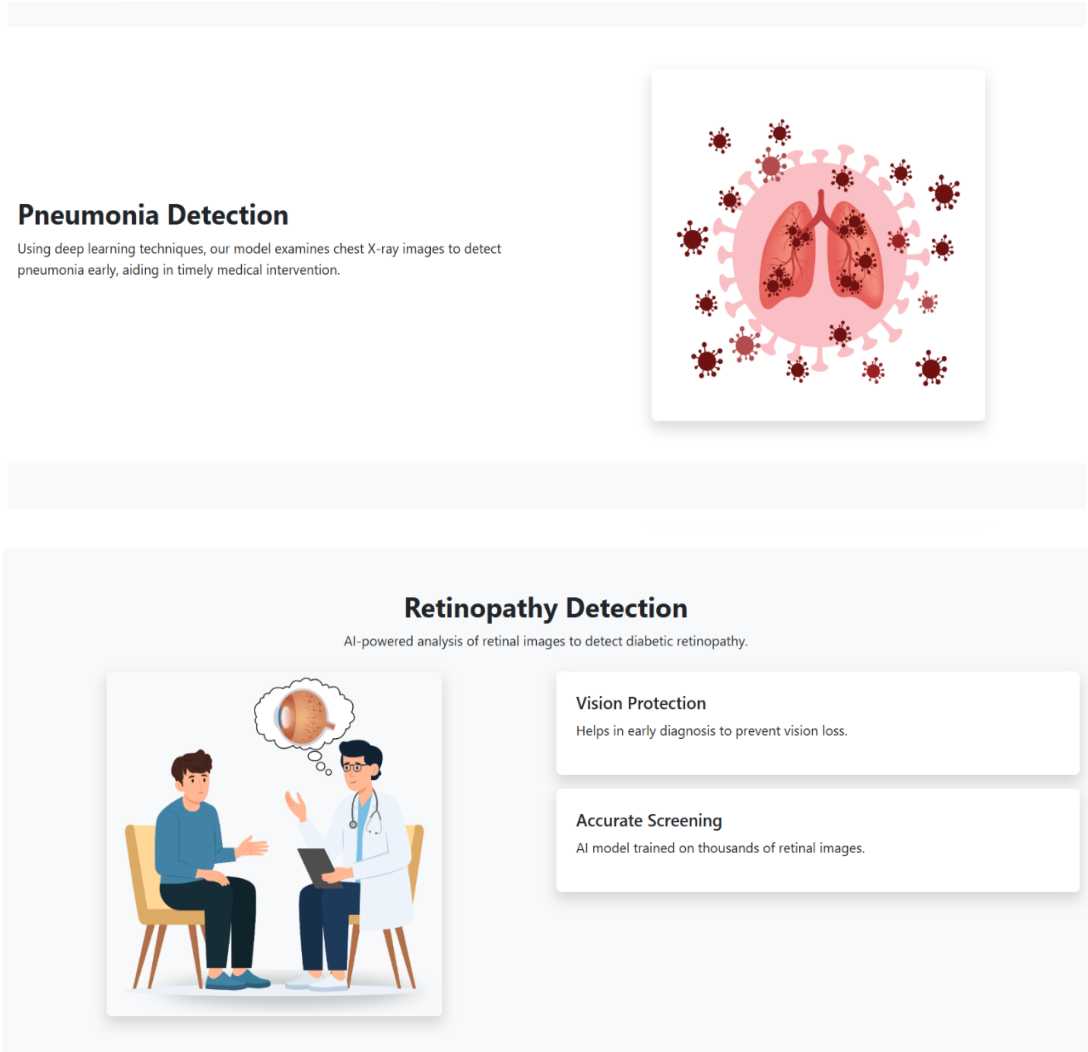

Fig 6.4.3: Services of the Project

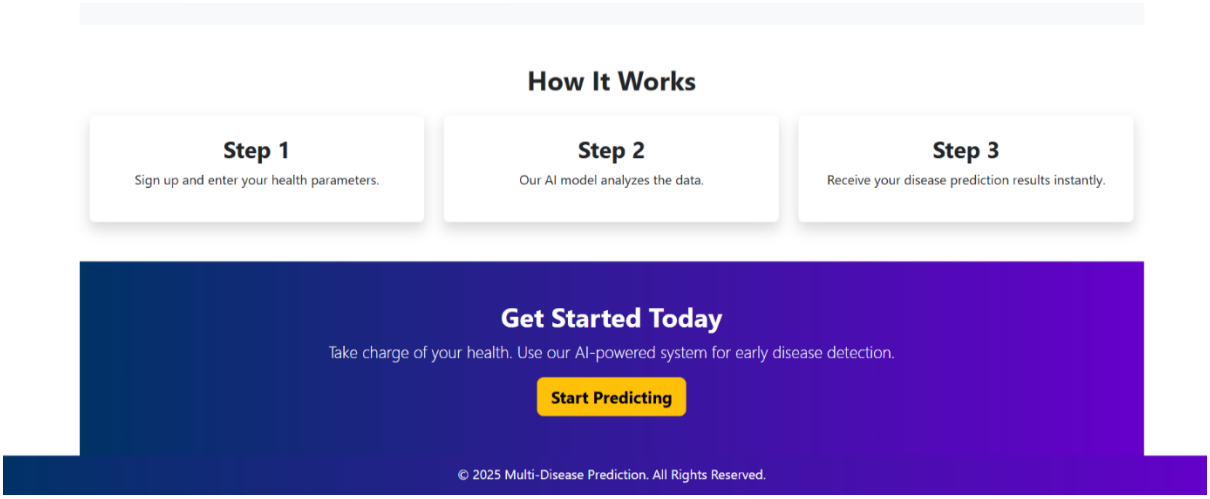

Fig 6.4.4: User guide to go through the services

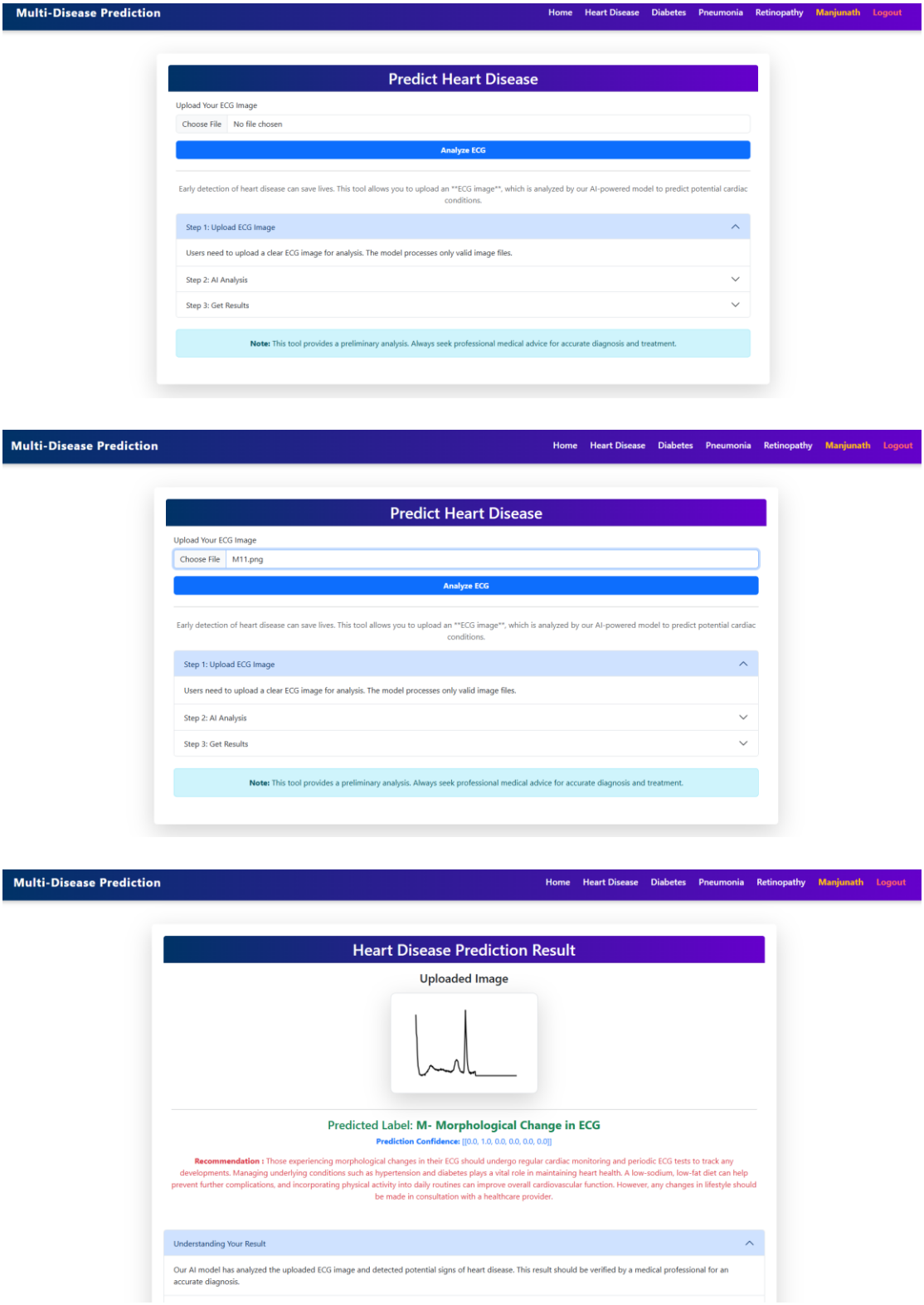

Fig 6.4.5: Heart Disease Prediction

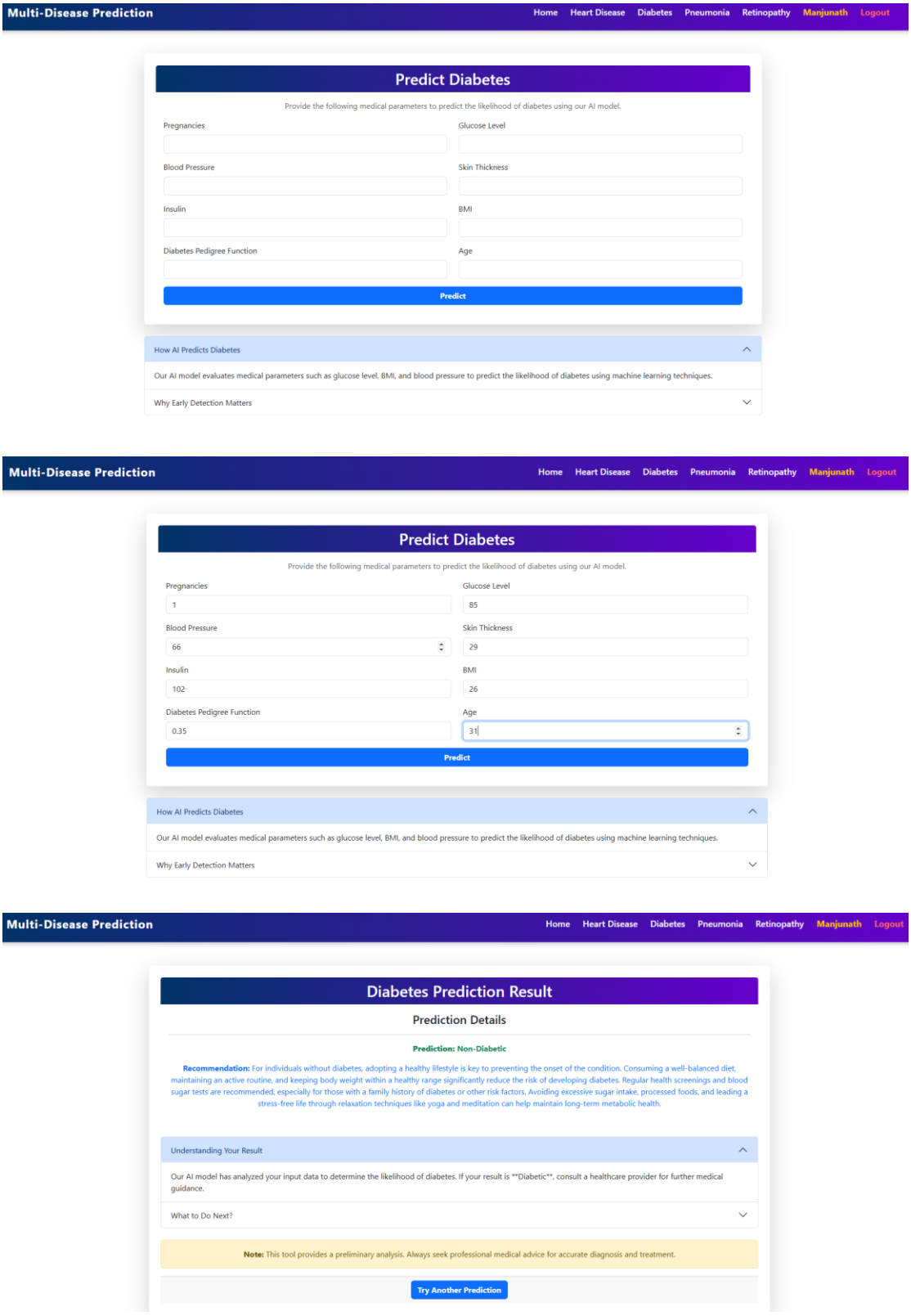

Fig 6.4.6: Diabetes Prediction

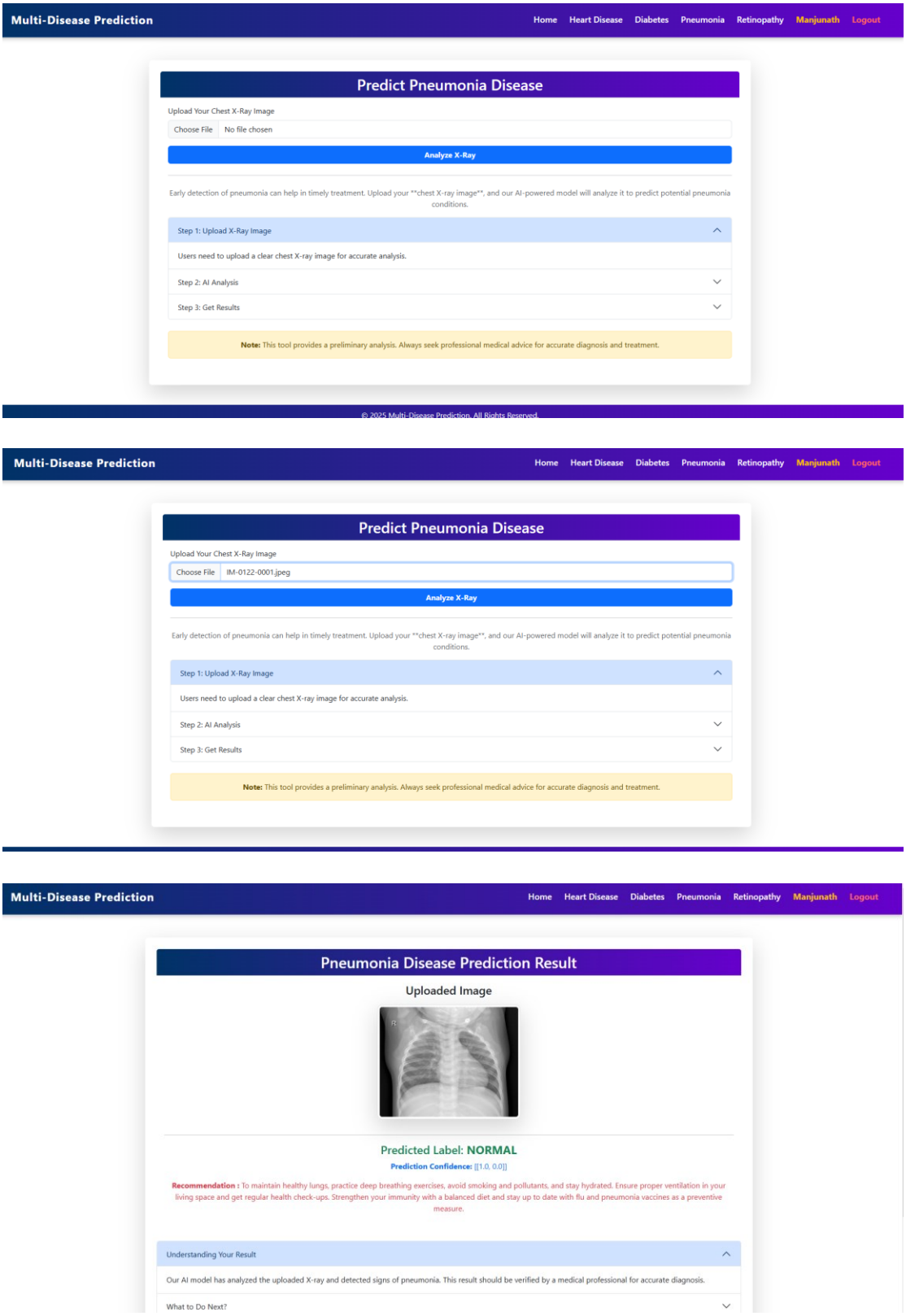

Fig 6.4.7: Pneumonia Prediction

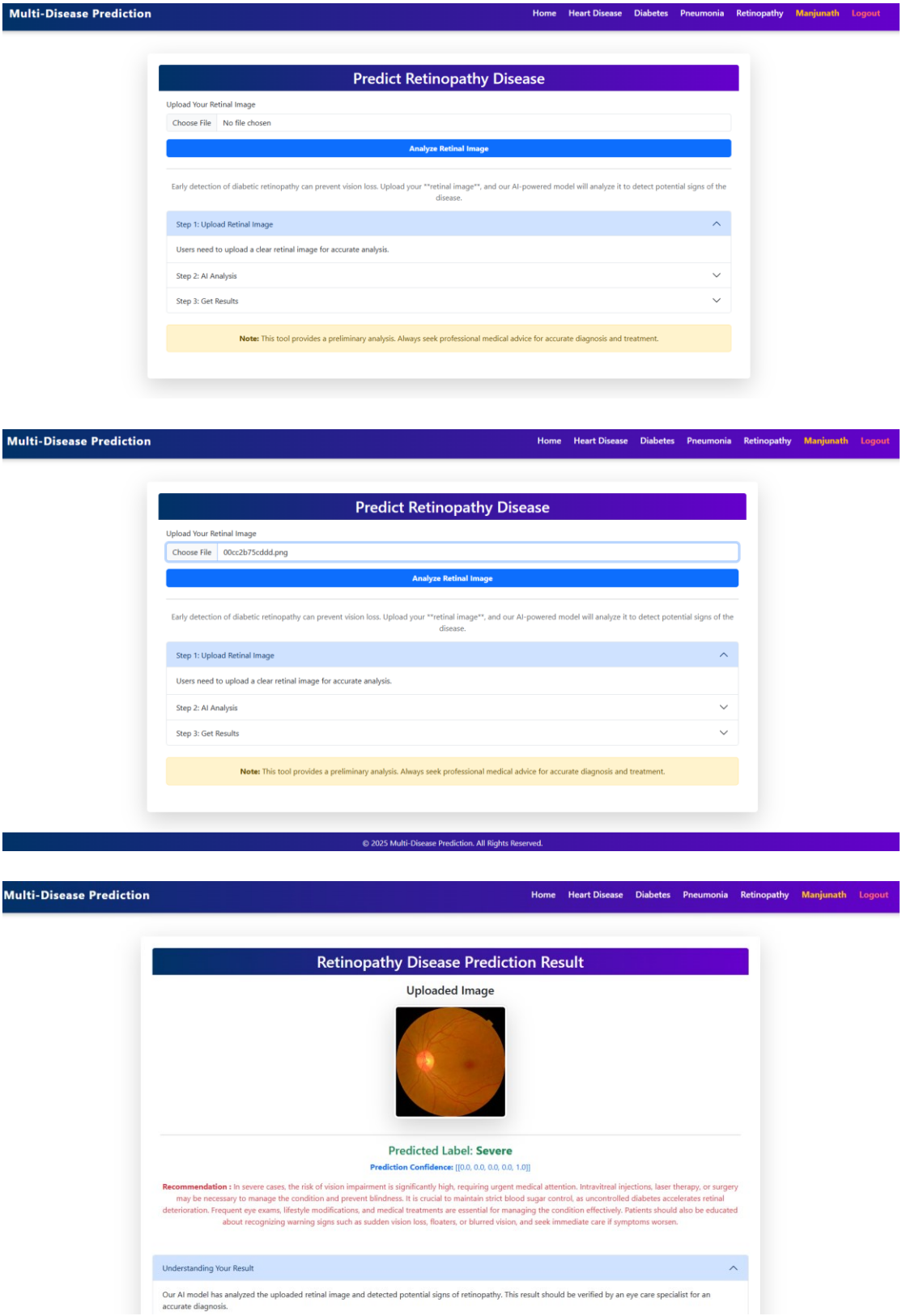

Fig 6.4.8: Diabetic Retinopathy Prediction

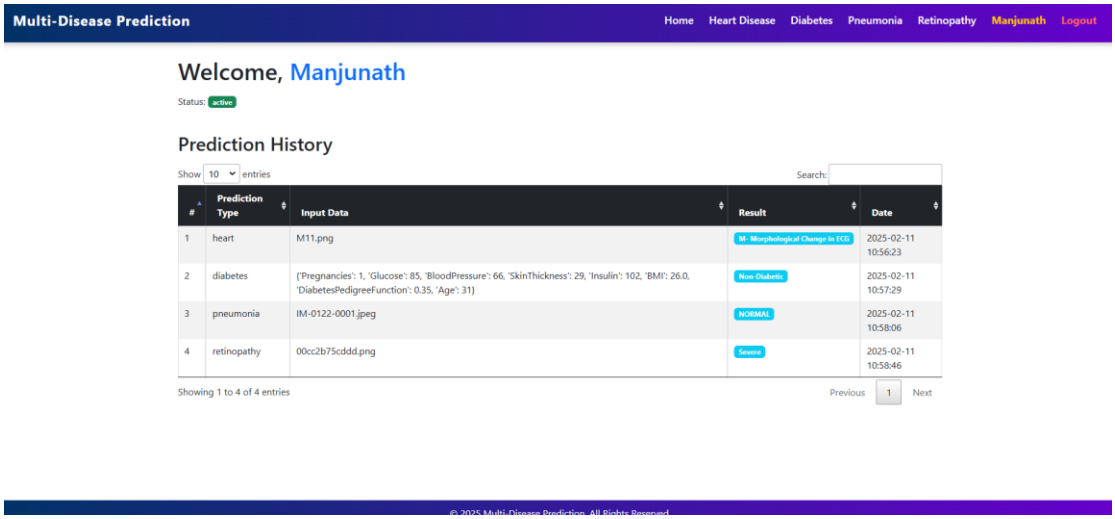

Fig 6.4.9: User Dashboard

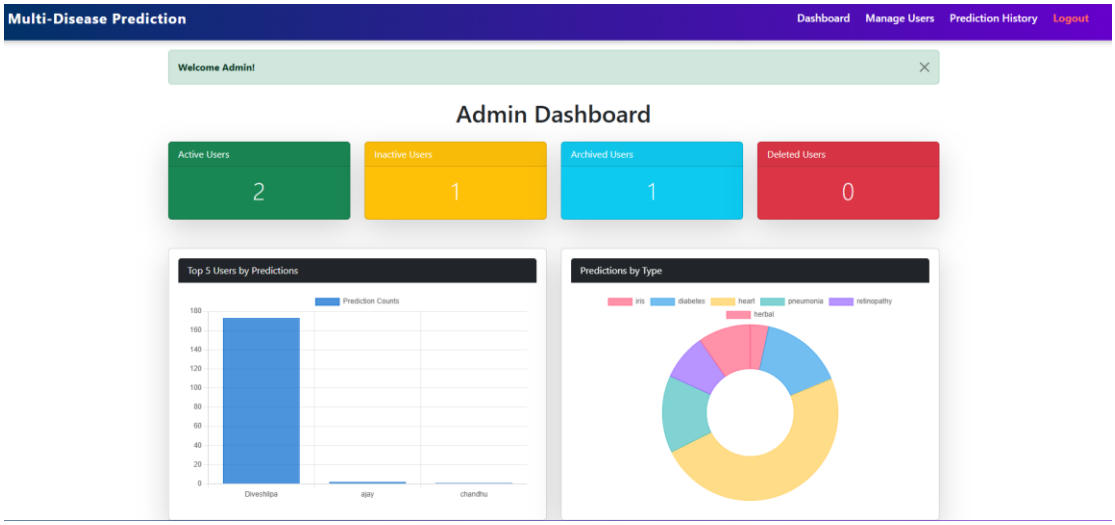

Fig 6.4.10: Admin Dashboard

Chapter 7: Conclusion and Future Work

7.1 Summary of Findings

This study demonstrates the effectiveness of deep learning in diagnosing heart disease, pneumonia, and diabetic retinopathy (DR) using CNN, ResNet, MobileNet, and DenseNet architectures. By leveraging diverse datasets such as the UCI Heart Disease dataset, chest X-rays, and fundus images, the proposed approach significantly enhances accuracy and computational efficiency. The pneumonia detection model, particularly using VGG16, achieves an impressive accuracy of 97%, outperforming traditional machine learning models like KNN, SVM, and Random Forest. The hybrid deep learning framework reduces computational cost and enhances real-time applicability in clinical settings. Comparative analysis further validates the model's robustness, making it a promising tool for automated disease diagnosis

7.2 Key Achievements and Contributions

Innovation:

The main innovation is the creation of a hybrid deep learning architecture that combines CNN, ResNet, MobileNet, and DenseNet models for concurrent diagnosis of heart disease, pneumonia, and diabetic retinopathy. This method seeks to enhance diagnostic accuracy and computational efficiency over conventional approaches.

Datasets Used: The system uses several diverse medical datasets:

- UCI Heart Disease dataset for classification of heart disease.
- Chest X-ray images for the classification of pneumonia.
- Color fundus images for DR classification.

Improvement in accuracy:

The neural network based on VGG16 yields a 97% accuracy in the classification of pneumonia. The hybrid approach as proposed exhibits better classification performance than the conventional machine learning algorithms such as KNN, SVM, Random Forest, and Naïve Bayes. The framework enhances real-time applicability in clinical settings by reducing computational costs and improving processing speed.

7.3 Challenges Faced

- Data representation variability poses difficulties for conventional diagnostic methods when processing heart disease, pneumonia and diabetic retinopathy datasets. The inconsistencies produce difficulties with model generalization thus requiring improved robust solutions for handling the problem.
- Modern machine learning systems lack effectiveness in extracting sophisticated features from medical picture data. Deep learning models together with Convolutional Neural Networks (CNNs) automatically discover important features for improved diagnostic outcomes.
- Machine learning techniques including K-Nearest Neighbors (KNN), Support Vector Machines (SVM) and Random Forest fail to perform deep feature extraction required for precise medical image diagnosis. The new system brings together innovative deep learning structures because they solve current system problems.
- The computational requirements of deep learning models run high because they need substantial processing power. The system utilizes MobileNet as its computationally efficient architecture while combining DenseNet and ResNet models for performance accuracy together with resource efficiency.
- Clinical applications require an immediate diagnosis process that should be available in real time. The system implements MobileNet as a lightweight architecture which allows quick processing while being suitable for real-time deployment in medical facilities.

7.4 Future Scope and Improvements

Future research can focus on expanding the dataset diversity to improve generalizability across different populations. Enhancing model interpretability using explainable AI techniques can increase clinical acceptance and trust. Integrating multi-modal medical data, including ECG signals and genomic information, may further refine diagnostic accuracy. Real-time implementation on edge devices such as mobile health applications can improve accessibility in remote areas. Additionally, optimizing deep learning architectures with efficient hyperparameter tuning can enhance both performance and inference speed. Collaborations with healthcare professionals for real-

world deployment and validation will be crucial in making this framework a standard diagnostic tool in medical practice.

Chapter 8: References

1. V. C A and V. Baby Shalini, "Systematic Review on Deep Learning-based Heart Disease Diagnosis," 2023 2nd International Conference on Edge Computing and Applications (ICECAA), Namakkal, India, 2023, pp. 908-912, doi: 10.1109/ICECAA58104.2023.10212392.

2. M. Z. Atwany, A. H. Sahyoun and M. Yaqub, "Deep Learning Techniques for Diabetic Retinopathy Classification: A Survey," in IEEE Access, vol. 10, pp. 28642-28655, 2022, doi: 10.1109/ACCESS.2022.3157632.

3. K. Mehta and K. Subramanian, "Heart Disease Diagnosis using Deep Learning," 2022 IEEE India Council International Subsections Conference (INDISCON), Bhubaneswar, India, 2022, pp. 1-6, doi: 10.1109/INDISCON54605.2022.9862847.

4. J. Wang, Y. Bai and B. Xia, "Simultaneous Diagnosis of Severity and Features of Diabetic Retinopathy in Fundus Photography Using Deep Learning," in IEEE Journal of Biomedical and Health Informatics, vol. 24, no. 12, pp. 3397-3407, Dec. 2020, doi: 10.1109/JBHI.2020.3012547

5. S. Qummar et al., "A Deep Learning Ensemble Approach for Diabetic Retinopathy Detection," in IEEE Access, vol. 7, pp. 150530-150539, 2019, doi: 10.1109/ACCESS.2019.2947484.